

**PERANCANGAN DAN IMPLEMENTASI SISTEM MANAJEMEN TUGAS BERBASIS
ARSITEKTUR MICROSERVICES NODE.JS, POSTGRESQL, DAN DOCKER**



Disusun oleh:

- 1. FEBRIANI DEWI ALVIAH (2310506010)**
- 2. VANIA AMELIA SETYA WIJAYA (2330506010)**

**PROGRAM STUDI S1 TEKNOLOGI INFORMASI
JURUSAN TEKNIK ELEKTRO MEKATRONIKA DAN INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS TIDAR**

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.....	5
1.1 Latar Belakang	5
1.2 Identifikasi Masalah	6
1.3 Rumusan Masalah	7
1.4 Tujuan Sistem.....	7
1.5 Manfaat Sistem.....	8
BAB II.....	9
2.1 Gambaran Umum	9
2.2 Analisis Permasalahan.....	10
2.3 Analisis Kebutuhan Fungsional	11
2.4 Analisis Non-Fungsional.....	12
2.4.1 Keamanan (Security)	12
2.4.2 Skalabilitas (Scalability).....	12
2.4.3 Maintainability.....	13
2.4.4 Availability	13
2.4.5 Reliability	13
2.4.6 Performance.....	13
2.5 Analisa Aktor Sistem	13
2.5.1 User.....	13
2.5.2 Sistem Otomatis.....	14
2.6 Analisis Alur Bisnis	14
BAB III	15
3.1 Use Case Diagram.....	15
3.1.1 Deskripsi Aktor.....	16
3.1.2 Deskripsi Use Case.....	16
3.2 Arsitektur Mikroservices.....	17
3.2.1 Struktur Arsitektur.....	18
3.2.2 Komunikasi Antar Service.....	18
3.3 Perancangan Basis Data (<i>Database Design</i>).....	19

3.3.1 User Database (user_db).....	19
3.3.2 Task Database (task_db).....	20
3.3.3 Notification Database (notif_db)	20
3.3.4 Hubungan Antar Entitas	21
3.4 Topologi Deployment	21
3.4.1 Lapisan Client.....	22
3.4.2 Lapisan API Gateway	22
3.4.3 Lapisan User Service	23
3.4.4 Lapisan Task Service.....	23
3.4.5 Lapisan Notification Service	23
3.4.6 Lapisan Database	24
3.4.7 Alur Komunikasi Sistem	24
3.5 Perancangan API (Application Programming Interface)	25
3.5.1 API User Service	25
3.5.2 API Task Service	26
3.5.3 API Notification Service	27
3.6 Perancangan JWT Authentication.....	28
3.7 Perancangan Docker Compose.....	29
3.8 Perancangan Dockerfile	30
3.8.1 Struktur Dockerfile User Service	31
3.8.2 Struktur Dockerfile Task Service	31
3.8.3 Struktur Dockerfile Notification Service.....	31
3.9 Kesimpulan Perancangan Sistem	32
BAB IV	33
4.1 Implementasi Sistem	33
4.2 Implementasi User Service.....	34
4.2.1 Implementasi Registrasi Pengguna.....	34
4.2.2 Implementasi Login.....	35
4.2.3 Implementasi Profil Pengguna.....	36
4.3 Implementasi Task Service	36
4.3.1 Implementasi Create Task	36
4.3.2 Implementasi Read Task	37

4.3.3 Implementasi Update Task	37
4.3.4 Implementasi Delete Task	38
4.3.5 Implementasi Assign Task.....	38
4.3.6 Implementasi Update Status Task	39
4.4 Implementasi Notification Service.....	39
4.4.1 Implementasi Pembuatan Notifikasi.....	40
4.4.2 Implementasi Menampilkan Notifikasi	40
4.4.3 Implementasi Menandai Notifikasi Telah Dibaca	40
4.5 Implementasi PostgreSQL Database	40
4.6 Implementasi API Gateway	45
4.7 Implementasi JWT Authentication.....	46
4.8 Implementasi Docker Compose	49
4.9 Pengujian Sistem	50
4.10 Analisis Hasil Pengujian	51
BAB V	53
5.1 Kesimpulan.....	53
5.2 Saran.....	54
DAFTAR PUSTAKA	56

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mendorong transformasi digital dalam berbagai aspek kehidupan, termasuk dalam pengelolaan pekerjaan dan koordinasi tugas pada lingkungan organisasi, perusahaan, maupun institusi pendidikan. Aktivitas pengelolaan tugas yang sebelumnya dilakukan secara manual kini mulai beralih menuju sistem digital yang mampu memberikan kemudahan dalam pencatatan, pemantauan, serta pengendalian pekerjaan secara lebih efektif dan efisien.

Pada praktiknya, banyak tim kerja masih mengandalkan media komunikasi seperti aplikasi pesan instan, surat elektronik, maupun catatan manual untuk mendistribusikan tugas kepada anggota tim. Meskipun metode tersebut relatif mudah digunakan, namun memiliki berbagai keterbatasan. Informasi tugas sering kali tersebar pada banyak percakapan sehingga sulit ditemukan kembali ketika dibutuhkan. Selain itu, tidak tersedia mekanisme pemantauan yang jelas mengenai status penyelesaian tugas sehingga pimpinan maupun anggota tim kesulitan mengetahui perkembangan pekerjaan secara real-time.

Permasalahan lainnya muncul ketika jumlah pengguna dan volume tugas meningkat. Sistem pengelolaan tugas yang dibangun menggunakan pendekatan monolitik cenderung mengalami kesulitan dalam proses pemeliharaan dan pengembangan. Setiap perubahan pada satu bagian sistem dapat mempengaruhi keseluruhan aplikasi sehingga meningkatkan risiko kegagalan layanan. Selain itu, proses deployment menjadi lebih kompleks karena seluruh komponen harus diperbarui secara bersamaan meskipun perubahan hanya dilakukan pada satu fitur tertentu.

Dalam konteks pengembangan perangkat lunak modern, kebutuhan akan sistem yang fleksibel, mudah dikembangkan, dan mampu beradaptasi terhadap perubahan menjadi semakin penting. Oleh karena itu, pendekatan arsitektur microservices menjadi salah satu solusi yang banyak digunakan. Microservices memungkinkan aplikasi dibagi menjadi beberapa layanan independen yang memiliki tanggung jawab spesifik. Setiap

layanan dapat dikembangkan, diuji, dan di-deploy secara terpisah tanpa mengganggu layanan lainnya.

Berdasarkan kondisi tersebut, dikembangkan sebuah Sistem Manajemen Tugas berbasis Microservices yang terdiri atas User Service, Task Service, dan Notification Service. Sistem ini dirancang untuk membantu proses pengelolaan tugas secara terstruktur, menyediakan mekanisme autentikasi yang aman, memungkinkan distribusi tugas kepada pengguna tertentu, serta memberikan notifikasi otomatis terkait aktivitas tugas yang terjadi dalam sistem.

Implementasi sistem berbasis microservices diharapkan dapat meningkatkan efisiensi pengelolaan tugas, mempermudah proses pengembangan sistem di masa mendatang, serta memberikan fondasi arsitektur yang lebih scalable dan maintainable dibandingkan pendekatan monolitik. Dengan demikian, sistem yang dibangun tidak hanya menyelesaikan permasalahan operasional saat ini, tetapi juga mampu mendukung kebutuhan pengembangan jangka panjang.

1.2 Identifikasi Masalah

Berdasarkan hasil analisis kebutuhan dan kondisi yang ditemukan pada proses pengelolaan tugas saat ini, dapat diidentifikasi beberapa permasalahan sebagai berikut:

1. Pengelolaan tugas masih dilakukan menggunakan media yang tidak terstruktur sehingga informasi mudah tercecer dan sulit dilacak.
2. Tidak tersedia mekanisme monitoring status tugas secara real-time sehingga perkembangan pekerjaan sulit diketahui oleh user maupun anggota tim.
3. Tidak terdapat sistem notifikasi yang mampu memberikan informasi otomatis ketika tugas baru diberikan kepada pengguna.
4. Pengelolaan data pengguna, tugas, dan notifikasi masih belum terintegrasi dalam satu sistem yang terpusat.
5. Sistem dengan pendekatan monolitik memiliki tingkat kompleksitas pemeliharaan yang tinggi dan sulit dikembangkan ketika kebutuhan sistem bertambah.
6. Skalabilitas aplikasi menjadi terbatas karena seluruh komponen sistem berjalan dalam satu kesatuan aplikasi yang saling bergantung.

7. Belum tersedia mekanisme autentikasi dan otorisasi yang terstandarisasi untuk mengatur hak akses pengguna berdasarkan perannya.

1.3 Rumusan Masalah

Berdasarkan identifikasi masalah yang telah dijabarkan sebelumnya, maka rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana merancang sistem manajemen tugas yang mampu mengelola proses distribusi dan pemantauan tugas secara efektif?
2. Bagaimana menerapkan arsitektur microservices pada sistem manajemen tugas agar lebih mudah dikembangkan dan dipelihara?
3. Bagaimana membangun mekanisme autentikasi dan otorisasi menggunakan JSON Web Token (JWT) untuk menjamin keamanan akses pengguna?
4. Bagaimana merancang komunikasi antar layanan menggunakan REST API sehingga setiap service dapat berinteraksi secara independen?
5. Bagaimana mengimplementasikan sistem notifikasi otomatis untuk meningkatkan efektivitas penyampaian informasi kepada pengguna?
6. Bagaimana menerapkan containerization menggunakan Docker untuk mempermudah deployment dan pengelolaan sistem?

1.4 Tujuan Sistem

Tujuan utama pembangunan Sistem Manajemen Tugas berbasis Microservices adalah sebagai berikut:

1. Mengembangkan sistem yang mampu mengelola tugas secara terstruktur dan terintegrasi.
2. Meningkatkan efektivitas distribusi tugas kepada pengguna melalui mekanisme assignment yang terpusat.
3. Menyediakan fasilitas pemantauan status tugas secara real-time.
4. Mengimplementasikan sistem notifikasi otomatis yang mampu memberikan informasi terkait perubahan tugas.
5. Menerapkan arsitektur microservices untuk meningkatkan modularitas, maintainability, dan scalability sistem.

6. Mengimplementasikan autentikasi dan otorisasi berbasis JWT guna meningkatkan keamanan sistem.
7. Mengintegrasikan Docker dan Docker Compose untuk mendukung proses deployment yang lebih efisien dan konsisten.

1.5 Manfaat Sistem

1. Bagi Pengguna

Sistem membantu pengguna dalam menerima, mengelola, dan memantau tugas yang diberikan secara lebih terstruktur. Pengguna juga memperoleh notifikasi otomatis sehingga tidak melewatkan informasi penting terkait tugas yang harus diselesaikan.

2. Bagi User

User memperoleh kemudahan dalam membuat tugas, mengassign tugas kepada pengguna tertentu, memantau progres pekerjaan, serta mengelola data pengguna melalui satu platform terintegrasi.

3. Bagi Pengembang

Sistem menjadi sarana implementasi konsep software engineering modern, khususnya arsitektur microservices, REST API, JWT Authentication, dan containerization menggunakan Docker. Pengembang juga dapat melakukan pengembangan fitur secara independen pada masing-masing layanan.

4. Bagi Organisasi

Organisasi memperoleh sistem yang lebih efisien dalam mendukung koordinasi pekerjaan, meningkatkan produktivitas anggota tim, serta menyediakan fondasi teknologi yang lebih fleksibel untuk pengembangan di masa mendatang.

BAB II

ANALISIS SISTEM

2.1 Gambaran Umum

Sistem Manajemen Tugas Berbasis Microservices merupakan sebuah aplikasi yang dirancang untuk membantu proses pengelolaan, distribusi, pemantauan, dan penyelesaian tugas dalam lingkungan kerja maupun pendidikan. Sistem ini dikembangkan untuk mengatasi berbagai permasalahan yang sering muncul pada pengelolaan tugas secara konvensional, seperti kesulitan dalam memantau progres pekerjaan, kurangnya koordinasi antar anggota tim, serta tidak adanya mekanisme notifikasi yang terintegrasi.

Secara umum, sistem memungkinkan user untuk membuat tugas, menentukan pengguna yang bertanggung jawab terhadap tugas tersebut, memantau status penyelesaian tugas, serta mengelola data pengguna yang terdaftar. Sementara itu, pengguna dapat melihat tugas yang diberikan kepadanya, memperbarui status pengerjaan tugas, serta menerima notifikasi terkait aktivitas yang terjadi pada sistem.

Dalam pengembangannya, sistem menerapkan pendekatan arsitektur microservices. Berbeda dengan sistem monolitik yang menggabungkan seluruh fungsi ke dalam satu aplikasi besar, microservices memecah aplikasi menjadi beberapa layanan independen yang memiliki tanggung jawab spesifik. Pendekatan ini memberikan fleksibilitas yang lebih tinggi dalam pengembangan, pengujian, deployment, dan pemeliharaan sistem.

Sistem terdiri atas tiga layanan utama yaitu User Service, Task Service, dan Notification Service. User Service bertanggung jawab terhadap proses autentikasi dan pengelolaan data pengguna. Task Service bertugas menangani seluruh proses yang berkaitan dengan manajemen tugas. Notification Service berfungsi menyediakan informasi notifikasi kepada pengguna ketika terjadi aktivitas tertentu dalam sistem.

Seluruh layanan tersebut dihubungkan melalui API Gateway yang bertindak sebagai pintu masuk tunggal (single entry point). API Gateway menerima seluruh permintaan dari client kemudian meneruskannya ke service yang sesuai berdasarkan endpoint yang diakses. Pendekatan ini memungkinkan pengelolaan autentikasi, routing,

dan keamanan dilakukan secara terpusat sehingga meningkatkan efisiensi dan konsistensi sistem.

Dari sisi infrastruktur, setiap service berjalan dalam container Docker yang terisolasi dan memiliki database PostgreSQL masing-masing. Implementasi ini mendukung prinsip loose coupling dan high cohesion yang menjadi karakteristik utama arsitektur microservices modern.

2.2 Analisis Permasalahan

Sebelum sistem dikembangkan, dilakukan analisis terhadap berbagai permasalahan yang sering muncul dalam proses pengelolaan tugas pada organisasi maupun kelompok kerja. Analisis ini bertujuan untuk mengidentifikasi akar masalah yang perlu diselesaikan melalui pembangunan sistem.

Permasalahan pertama adalah proses distribusi tugas yang masih dilakukan melalui media komunikasi umum seperti aplikasi pesan instan. Informasi tugas sering kali tercampur dengan percakapan lain sehingga sulit ditemukan kembali ketika diperlukan. Kondisi ini menyebabkan risiko kehilangan informasi serta menurunkan efektivitas koordinasi antar anggota tim.

Permasalahan kedua adalah tidak tersedianya mekanisme pemantauan status tugas secara real-time. Dalam banyak kasus, user harus menanyakan secara manual perkembangan pekerjaan kepada setiap anggota tim. Proses ini tidak efisien dan berpotensi menimbulkan keterlambatan dalam penyelesaian tugas karena kurangnya visibilitas terhadap progres pekerjaan.

Permasalahan berikutnya berkaitan dengan sistem notifikasi. Pada metode konvensional, pengguna sering tidak menyadari adanya tugas baru yang diberikan karena tidak terdapat sistem yang secara otomatis mengirimkan pemberitahuan. Akibatnya, tugas dapat terlambat dikerjakan atau bahkan terlewatkan.

Dari sisi teknis, penggunaan sistem monolitik memiliki beberapa keterbatasan. Ketika jumlah pengguna meningkat atau fitur baru ditambahkan, kompleksitas aplikasi menjadi semakin tinggi. Perubahan kecil pada satu modul dapat mempengaruhi modul lainnya sehingga meningkatkan risiko terjadinya kegagalan sistem.

Selain itu, proses deployment pada sistem monolitik cenderung kurang fleksibel. Setiap pembaruan memerlukan deployment ulang seluruh aplikasi meskipun perubahan hanya dilakukan pada satu fitur tertentu. Kondisi ini menyebabkan waktu pemeliharaan menjadi lebih lama dan meningkatkan kemungkinan terjadinya downtime.

Berdasarkan analisis tersebut, diperlukan sebuah sistem yang mampu mengelola tugas secara terstruktur, menyediakan pemantauan progres secara real-time, mendukung notifikasi otomatis, serta memiliki arsitektur yang fleksibel dan mudah dikembangkan. Oleh karena itu, dipilihlah pendekatan microservices sebagai solusi utama dalam pengembangan sistem.

2.3 Analisis Kebutuhan Fungsional

Kebutuhan fungsional merupakan kebutuhan yang menjelaskan layanan atau fungsi yang harus disediakan oleh sistem agar dapat memenuhi kebutuhan pengguna.

1. Fungsi pertama yang harus tersedia adalah registrasi pengguna. Sistem harus memungkinkan pengguna baru untuk membuat akun dengan memasukkan informasi yang diperlukan seperti username, email, dan password. Data yang berhasil didaftarkan akan disimpan pada User Service untuk digunakan dalam proses autentikasi.
2. Fungsi kedua adalah login pengguna. Setelah akun berhasil dibuat, pengguna dapat melakukan autentikasi menggunakan username dan password. Jika data valid, sistem akan menghasilkan token JWT yang digunakan untuk mengakses layanan lain dalam sistem.
3. Fungsi ketiga adalah manajemen profil pengguna. Pengguna dapat melihat informasi profilnya sendiri, sedangkan user memiliki hak akses tambahan untuk melihat seluruh pengguna yang terdaftar dalam sistem.
4. Fungsi keempat adalah pengelolaan tugas. User dapat membuat tugas baru dengan memasukkan informasi seperti judul tugas, deskripsi tugas, tenggat waktu, dan pengguna yang akan menerima tugas tersebut. Data tugas kemudian disimpan pada Task Service.
5. Fungsi kelima adalah pengubahan status tugas. Pengguna yang menerima tugas dapat memperbarui status tugas sesuai perkembangan pekerjaannya, misalnya dari status pending menjadi done.

6. Fungsi keenam adalah penghapusan tugas. User memiliki hak untuk menghapus tugas yang sudah tidak diperlukan lagi dari sistem.
7. Fungsi ketujuh adalah proses assignment tugas. User dapat menetapkan tugas kepada pengguna tertentu sehingga tanggung jawab pekerjaan menjadi jelas dan terdokumentasi.
8. Fungsi kedelapan adalah pembuatan notifikasi otomatis. Ketika user melakukan assignment tugas, sistem secara otomatis membuat notifikasi melalui Notification Service.
9. Fungsi kesembilan adalah melihat daftar notifikasi. Pengguna dapat mengakses seluruh notifikasi yang dimilikinya melalui endpoint yang tersedia.
10. Fungsi kesepuluh adalah menandai notifikasi sebagai telah dibaca sehingga pengguna dapat membedakan notifikasi yang sudah dan belum diperiksa.

Secara keseluruhan, kebutuhan fungsional tersebut membentuk proses bisnis utama sistem mulai dari autentikasi pengguna hingga pengelolaan tugas dan notifikasi.

2.4 Analisis Non-Fungsional

Selain kebutuhan fungsional, sistem juga harus memenuhi kebutuhan non-fungsional yang berkaitan dengan kualitas layanan yang diberikan.

2.4.1 Keamanan (Security)

Keamanan merupakan aspek penting karena sistem mengelola data pengguna dan tugas yang bersifat privat. Sistem menerapkan mekanisme autentikasi menggunakan JSON Web Token (JWT) sehingga setiap pengguna harus melakukan login terlebih dahulu sebelum mengakses layanan tertentu. Selain itu, otorisasi berbasis role diterapkan untuk membedakan hak akses antara user dan pengguna biasa.

2.4.2 Skalabilitas (Scalability)

Sistem dirancang menggunakan arsitektur microservices sehingga setiap service dapat ditingkatkan kapasitasnya secara independen. Jika jumlah pengguna meningkat, hanya service yang mengalami beban tinggi yang perlu diperluas tanpa mempengaruhi layanan lainnya.

2.4.3 Maintainability

Pemisahan sistem ke dalam beberapa *service independen* membuat proses pemeliharaan menjadi lebih mudah. Setiap service memiliki kode program, *database*, dan *deployment* sendiri sehingga perubahan pada satu service tidak mempengaruhi service lainnya.

2.4.4 Availability

Availability menunjukkan kemampuan sistem untuk tetap tersedia ketika dibutuhkan pengguna. Dengan menggunakan *container Docker* dan pemisahan layanan, kegagalan pada satu service tidak secara langsung menyebabkan seluruh sistem berhenti beroperasi.

2.4.5 Reliability

Reliability berkaitan dengan kemampuan sistem menjalankan fungsi secara konsisten. Penggunaan PostgreSQL sebagai DBMS memberikan dukungan terhadap transaksi data yang andal sehingga mengurangi risiko inkonsistensi data.

2.4.6 Performance

Kinerja sistem ditingkatkan melalui distribusi beban kerja ke beberapa service yang berbeda. Setiap service hanya menangani tanggung jawab tertentu sehingga penggunaan sumber daya menjadi lebih efisien dibandingkan sistem monolitik.

2.5 Analisa Aktor Sistem

Dalam sistem yang dikembangkan terdapat tiga aktor utama yang terlibat dalam proses bisnis.

2.5.1 User

User merupakan pengguna dengan tingkat hak akses tertinggi. User memiliki kewenangan untuk membuat tugas, menghapus tugas, mengassign tugas kepada pengguna tertentu, melihat daftar pengguna, serta memantau perkembangan seluruh tugas yang ada dalam sistem. User bertanggung jawab dalam mengelola operasional sistem dan memastikan setiap tugas didistribusikan kepada pengguna yang tepat. Oleh karena itu, user menjadi aktor yang paling banyak berinteraksi dengan *Task Service*.

2.5.2 Sistem Otomatis

Selain aktor manusia, terdapat aktor berupa sistem otomatis yang bekerja tanpa intervensi pengguna. Aktor ini bertanggung jawab menghasilkan notifikasi ketika terjadi peristiwa tertentu, seperti assignment tugas baru atau perubahan status tugas. Keberadaan sistem otomatis membantu meningkatkan responsivitas aplikasi karena pengguna dapat memperoleh informasi secara langsung tanpa perlu melakukan pengecekan manual.

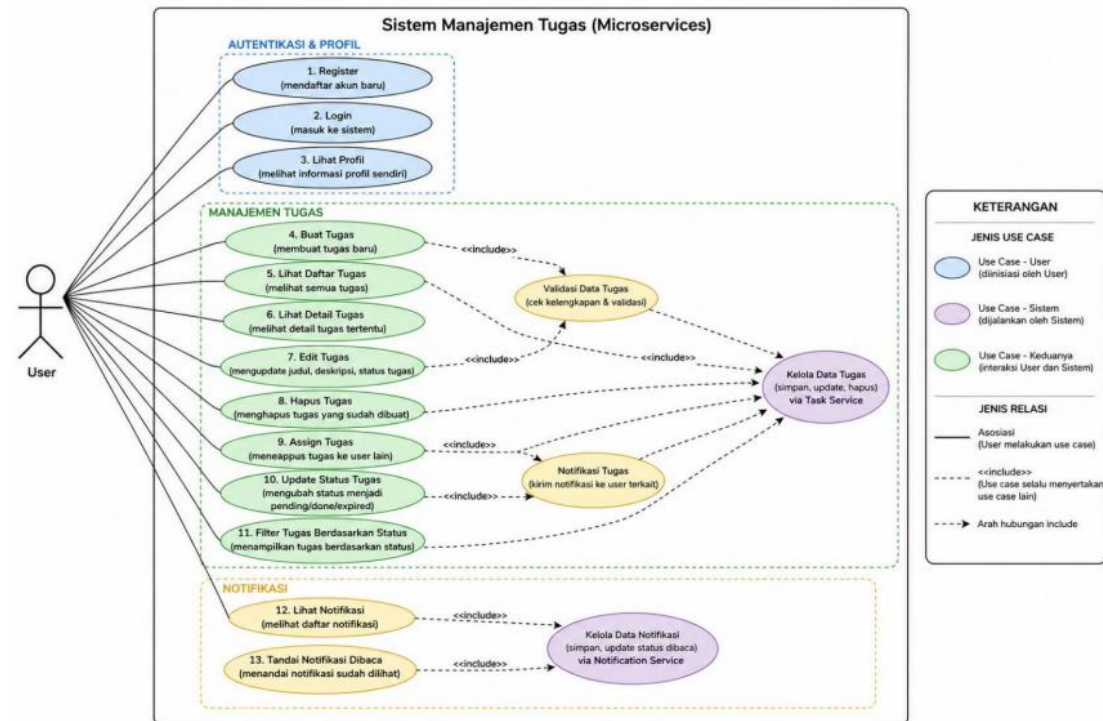
2.6 Analisis Alur Bisnis

1. Alur bisnis sistem dimulai ketika pengguna melakukan registrasi akun melalui User Service. Data yang dimasukkan pengguna akan diverifikasi dan disimpan ke dalam database pengguna. Setelah berhasil melakukan registrasi, pengguna dapat masuk ke sistem melalui proses login.
2. Pada saat login berhasil dilakukan, sistem menghasilkan token JWT yang digunakan sebagai identitas digital pengguna selama sesi berlangsung. Token tersebut akan dikirimkan pada setiap permintaan ke layanan lain melalui API Gateway.
3. User kemudian dapat membuat tugas baru melalui Task Service. Data tugas yang dimasukkan mencakup judul tugas, deskripsi, tenggat waktu, serta pengguna yang akan menerima tugas tersebut. Setelah data tersimpan, *Task Service* akan mengirimkan permintaan ke *Notification Service* untuk membuat notifikasi baru.
4. Notification Service menyimpan notifikasi ke dalam database dan menyediakan *endpoint* yang memungkinkan pengguna melihat daftar notifikasi yang dimilikinya. Dengan demikian, pengguna dapat segera mengetahui adanya tugas baru yang diberikan.
5. Setelah menerima tugas, pengguna dapat melihat detail tugas melalui *Task Service* dan mulai mengerjakannya. Ketika terdapat perkembangan pekerjaan, pengguna dapat memperbarui status tugas sesuai kondisi aktual, misalnya dari pending menjadi *in progress* atau *done*.
6. Seluruh perubahan yang terjadi pada tugas akan tersimpan pada *database Task Service* dan dapat dipantau oleh user. Dengan alur bisnis tersebut, proses distribusi, pelaksanaan, pemantauan, dan penyelesaian tugas dapat berlangsung secara terintegrasi dalam satu ekosistem sistem informasi yang modern dan scalable.

BAB III

PERANCANGAN SISTEM

3.1 Use Case Diagram



Use Case Diagram digunakan untuk menggambarkan interaksi antara pengguna dengan sistem yang dikembangkan. Diagram ini menunjukkan fungsi-fungsi utama yang tersedia serta hak akses yang dimiliki oleh masing-masing aktor. Pada Sistem Manajemen Tugas Berbasis Microservices terdapat dua aktor utama, yaitu User. Selain itu terdapat proses otomatis yang dijalankan sistem untuk menghasilkan notifikasi ketika terjadi aktivitas tertentu pada tugas.

User merupakan pengguna yang memiliki hak akses penuh terhadap sistem. User dapat melakukan login, melihat daftar pengguna, membuat tugas baru, mengubah data tugas, menghapus tugas, serta mengassign tugas kepada pengguna tertentu. Dengan kewenangan tersebut, user bertanggung jawab dalam mengelola seluruh aktivitas tugas yang terdapat pada sistem.

User merupakan pengguna yang menerima tugas dari user. Setelah berhasil melakukan login, pengguna dapat melihat daftar tugas yang diberikan kepadanya, memperbarui status tugas sesuai progres pekerjaan, melihat notifikasi yang diterima, serta menandai notifikasi yang telah dibaca.

Selain aktor user tersebut, sistem juga memiliki mekanisme otomatis yang bekerja ketika user melakukan assignment tugas kepada pengguna. Pada kondisi tersebut, sistem akan memicu Notification Service untuk membuat notifikasi baru yang kemudian dapat dilihat oleh pengguna yang bersangkutan.

Berdasarkan rancangan sistem, *use case* utama yang tersedia meliputi *Login*, *View Users*, *Create Task*, *Update Task*, *Delete Task*, *Assign Task*, *View Task List*, *Update Task Status*, *View Notification*, dan *Mark Notification as Read*. Seluruh *use case* tersebut saling terintegrasi untuk mendukung proses pengelolaan tugas secara terstruktur.

3.1.1 Deskripsi Aktor

1. User

User memiliki hak akses untuk:

- a Login ke sistem
- b Melihat daftar seluruh pengguna
- c Membuat tugas baru
- d Mengubah data tugas
- e Menghapus tugas
- f Mengassign tugas kepada pengguna
- g Melihat daftar tugas
- h Mengubah status tugas
- i Melihat notifikasi
- j Menandai notifikasi sebagai telah dibaca

3.1.2 Deskripsi Use Case

Use Case	Aktor	Deskripsi
<i>Login</i>	User	Masuk ke sistem menggunakan akun terdaftar

<i>View Users</i>	User	Melihat seluruh data pengguna
<i>Create Task</i>	User	Membuat tugas baru
<i>Update Task</i>	User	Mengubah informasi tugas
<i>Delete Task</i>	User	Menghapus tugas
<i>Assign Task</i>	User	Menugaskan tugas kepada pengguna
<i>View Task List</i>	User	Melihat daftar tugas
<i>Update Task Status</i>	User	Mengubah status penyelesaian
<i>View Notification</i>	User	Melihat daftar notifikasi
<i>Mark Notification as Read</i>	User	Menandai notifikasi telah di baca

3.2 Arsitektur Mikroservices

Sistem Manajemen Tugas dirancang menggunakan arsitektur microservices untuk meningkatkan fleksibilitas pengembangan, skalabilitas sistem, serta kemudahan pemeliharaan aplikasi. Berbeda dengan pendekatan monolitik yang menggabungkan seluruh fungsi ke dalam satu aplikasi besar, microservices memisahkan sistem menjadi beberapa layanan independen yang dapat berjalan secara terpisah.

Pada rancangan ini, sistem dibagi menjadi tiga layanan utama yaitu *User Service*, *Task Service*, dan *Notification Service*. Masing-masing service memiliki tanggung jawab bisnis yang berbeda sehingga perubahan pada satu service tidak akan mempengaruhi service lainnya secara langsung.

User Service bertanggung jawab terhadap pengelolaan pengguna dan autentikasi sistem. Seluruh proses login serta pengelolaan data pengguna dilakukan pada *service* ini. Dengan pemisahan tersebut, proses autentikasi dapat dikelola secara terpusat dan lebih mudah dikembangkan.

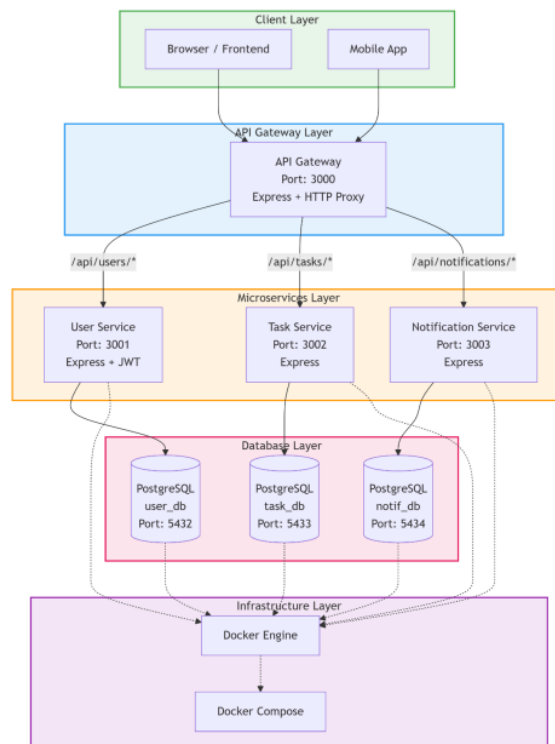
Task Service berfungsi sebagai layanan utama yang menangani seluruh aktivitas pengelolaan tugas. Service ini bertanggung jawab terhadap proses pembuatan tugas, pembaruan tugas, penghapusan tugas, *assignment* tugas, serta perubahan status tugas oleh pengguna.

Notification Service berfungsi menyediakan informasi kepada pengguna terkait aktivitas yang terjadi dalam sistem. Ketika sebuah tugas diassign kepada pengguna tertentu, *Task Service* akan mengirimkan data ke *Notification Service* untuk membuat notifikasi baru.

Untuk mempermudah akses client, seluruh request tidak langsung dikirim ke service terkait melainkan melalui API Gateway. API Gateway bertindak sebagai pintu masuk tunggal yang menerima request dari client kemudian meneruskannya ke *service* yang sesuai berdasarkan *endpoint* yang diakses.

Pendekatan ini memberikan sejumlah keuntungan, antara lain meningkatkan modularitas sistem, mempermudah proses deployment, mendukung skalabilitas independen, serta mengurangi dampak kegagalan apabila salah satu service mengalami masalah.

3.2.1 Struktur Arsitektur



3.2.2 Komunikasi Antar Service

Komunikasi antar service dilakukan menggunakan REST API berbasis HTTP.

Contoh alur:

1. User membuat tugas.
2. Data tugas disimpan oleh *Task Service*.
3. *Task Service* mengirim *request* ke *Notification Service*.
4. *Notification Service* membuat data notifikasi.
5. Pengguna menerima notifikasi tugas baru.

3.3 Perancangan Basis Data (*Database Design*)

Perancangan basis data pada sistem menggunakan prinsip *Database per Service*. Setiap service memiliki *database* PostgreSQL yang terpisah sehingga tidak terdapat akses langsung antar *database*. Pendekatan ini merupakan salah satu karakteristik utama arsitektur microservices karena memungkinkan setiap service mengelola data secara independen. Selain meningkatkan keamanan, pemisahan *database* juga mempermudah proses pemeliharaan dan pengembangan sistem.

3.3.1 User Database (*user_db*)

Database *user_db* digunakan oleh *User Service* untuk menyimpan informasi pengguna yang terdaftar pada sistem.

Tabel	Kolom	Tipe	Keterangan
users	id	SERIAL (PK)	ID unik user
	username	VARCHAR(100)	Nama pengguna
	email	VARCHAR(255) UNIQUE	Email (unique)
	password	VARCHAR(255)	Password terhash
	created_at	TIMESTAMP	Waktu registrasi

Primary key:

- Id

Fungsi tabel ini adalah menyimpan identitas pengguna yang akan digunakan dalam proses autentikasi dan otorisasi sistem.

3.3.2 Task Database (task_db)

Database task_db digunakan oleh *Task Service* untuk menyimpan seluruh data tugas.

Tabel	Kolom	Tipe	Keterangan
tasks	id	SERIAL (PK)	ID unik tugas
	title	VARCHAR(255)	Judul tugas
	description	TEXT	Deskripsi tugas
	status	VARCHAR(50)	pending/done/expired
	assigned_to	INTEGER (FK)	ID user penerima tugas
	created_by	INTEGER (FK)	ID user pembuat tugas
	created_at	TIMESTAMP	Waktu dibuat
	updated_at	TIMESTAMP	Waktu terakhir update

Primary key:

- Id

Kolom *assigned_to* digunakan untuk menyimpan identitas pengguna yang menerima tugas, sedangkan *created_by* digunakan untuk menyimpan identitas user yang membuat tugas.

3.3.3 Notification Database (notif_db)

Database notif_db digunakan oleh Notification Service untuk menyimpan data notifikasi pengguna.

Tabel	Kolom	Tipe	Keterangan
notifications	id	SERIAL (PK)	ID unik notifikasi
	user_id	INTEGER (FK)	ID user penerima
	title	VARCHAR(255)	Judul notifikasi
	message	TEXT	Isi notifikasi
	is_read	BOOLEAN	Status baca (default false)
	created_at	TIMESTAMP	Waktu dibuat

Primary Key:

- Id

Tabel ini digunakan untuk menyimpan seluruh aktivitas notifikasi yang dihasilkan sistem.

3.3.4 Hubungan Antar Entitas

Secara konseptual hubungan antar entitas adalah sebagai berikut:

1. Satu pengguna dapat menerima banyak tugas.
2. Satu useri dapat membuat banyak tugas.
3. Satu pengguna dapat menerima banyak notifikasi.
4. Satu tugas dapat menghasilkan satu notifikasi atau lebih.

Hubungan tersebut diimplementasikan melalui pertukaran data antar service menggunakan REST API tanpa adanya foreign key lintas database.

3.4 Topologi Deployment

Topologi deployment merupakan rancangan infrastruktur yang menggambarkan bagaimana seluruh komponen sistem ditempatkan, dijalankan, dan saling berkomunikasi pada lingkungan operasional. Pada Sistem Manajemen Tugas Berbasis Microservices, deployment dilakukan menggunakan teknologi *containerization* melalui Docker dan Docker Compose. Pendekatan ini dipilih karena mampu menyediakan lingkungan eksekusi yang konsisten antara tahap pengembangan, pengujian, dan implementasi sistem.

Penggunaan Docker memungkinkan setiap komponen aplikasi dijalankan dalam container yang terisolasi. Dengan mekanisme tersebut, setiap layanan memiliki lingkungan kerja yang independen sehingga konflik dependensi antar aplikasi dapat dihindari. Selain itu, proses deployment menjadi lebih sederhana karena seluruh kebutuhan aplikasi telah dikemas dalam image Docker yang dapat dijalankan pada berbagai platform tanpa memerlukan konfigurasi ulang yang kompleks.

Mengelola seluruh container secara terpusat, sistem menggunakan Docker Compose. File `docker-compose.yml` berfungsi sebagai orkestrator yang mendefinisikan seluruh layanan, database, konfigurasi jaringan, volume penyimpanan, serta dependensi antar container. Melalui pendekatan ini, seluruh sistem dapat dijalankan hanya dengan satu perintah, yaitu `docker-compose up --build`.

Topologi deployment yang dirancang terdiri atas tujuh container utama, yaitu *API Gateway*, *User Service*, *Task Service*, *Notification Service*, *User Database*, *Task Database*, dan *Notification Database*. Seluruh container tersebut dihubungkan melalui jaringan Docker bernama *task-network* yang menggunakan *driver bridge*. Dengan konfigurasi tersebut, setiap service dapat berkomunikasi menggunakan nama container sebagai *hostname* tanpa memerlukan konfigurasi alamat IP secara manual.

3.4.1 Lapisan Client

Lapisan client merupakan komponen yang berinteraksi secara langsung dengan pengguna. Client dapat berupa aplikasi web, aplikasi mobile, Postman, maupun aplikasi *frontend* lainnya yang memanfaatkan layanan REST API. Seluruh permintaan dari client tidak dikirimkan langsung ke masing-masing microservice, melainkan diarahkan terlebih dahulu menuju API Gateway. Pendekatan ini bertujuan untuk menyederhanakan akses layanan, meningkatkan keamanan sistem, serta mengurangi kompleksitas komunikasi antara client dan backend. Pengguna hanya perlu mengetahui satu alamat akses utama yang disediakan oleh API Gateway. Dengan demikian, perubahan struktur internal microservices tidak akan mempengaruhi aplikasi client yang telah menggunakan sistem.

3.4.2 Lapisan API Gateway

API Gateway berperan sebagai pintu masuk tunggal (*single entry point*) bagi seluruh permintaan yang berasal dari client. Komponen ini berjalan pada port 3000 dan bertugas melakukan routing permintaan menuju service yang sesuai berdasarkan endpoint yang diakses. Selain berfungsi sebagai reverse proxy, API Gateway juga dapat digunakan untuk melakukan validasi token JWT secara terpusat sebelum permintaan diteruskan ke service tujuan. Pendekatan ini membantu mengurangi duplikasi proses autentikasi pada setiap service sehingga pengelolaan keamanan menjadi lebih efisien. Konfigurasi routing pada API Gateway dibagi menjadi beberapa jalur utama. Endpoint autentikasi dan pengguna diteruskan menuju User Service, endpoint tugas diteruskan menuju Task Service, sedangkan endpoint notifikasi diteruskan menuju Notification Service. Dengan mekanisme tersebut, client hanya berkomunikasi dengan satu endpoint publik tanpa perlu mengetahui lokasi masing-masing service.

3.4.3 Lapisan User Service

User Service merupakan layanan yang bertanggung jawab terhadap proses manajemen pengguna dan autentikasi sistem. Service ini berjalan pada port 3001 dan terhubung dengan database `user_db` yang berada pada container PostgreSQL tersendiri. Fungsi utama User Service meliputi registrasi pengguna, proses login, pengelolaan profil pengguna, serta pembuatan token autentikasi menggunakan JSON Web Token (JWT). Ketika pengguna berhasil melakukan login, service akan menghasilkan token yang digunakan sebagai identitas pengguna saat mengakses layanan lainnya. Pemisahan User Service ke dalam container tersendiri memberikan keuntungan berupa kemudahan pemeliharaan serta fleksibilitas pengembangan fitur autentikasi tanpa mempengaruhi service lainnya.

3.4.4 Lapisan Task Service

Task Service merupakan layanan inti dalam sistem yang bertanggung jawab terhadap pengelolaan tugas. Service ini berjalan pada port 3002 dan menggunakan database `task_db` sebagai media penyimpanan data tugas. Seluruh proses CRUD (*Create, Read, Update, Delete*) tugas dilakukan pada service ini. Selain itu, Task Service juga bertanggung jawab dalam proses *assignment* tugas kepada pengguna tertentu serta pembaruan status penyelesaian tugas. Ketika user melakukan *assignment* tugas, *Task Service* akan mengirimkan permintaan HTTP POST kepada *Notification Service* untuk menghasilkan notifikasi baru. Mekanisme tersebut menunjukkan implementasi komunikasi antar service dalam arsitektur microservices yang dibangun.

3.4.5 Lapisan Notification Service

Notification Service bertugas menyediakan layanan notifikasi bagi pengguna. Service ini berjalan pada port 3003 dan menggunakan database `notif_db` untuk menyimpan seluruh informasi notifikasi yang dihasilkan sistem. Layanan ini menyediakan endpoint untuk melihat daftar notifikasi pengguna, menandai notifikasi sebagai telah dibaca, serta menerima permintaan internal dari Task Service ketika terjadi *assignment* tugas. Dengan adanya service ini, proses penyampaian informasi kepada pengguna dapat dilakukan secara otomatis dan terstruktur. Pemisahan layanan notifikasi menjadi service independen juga memudahkan pengembangan fitur di masa

mendatang, seperti integrasi email, *push notification*, maupun layanan pesan instan tanpa mengubah struktur *Task Service*.

3.4.6 Lapisan Database

Setiap microservice memiliki database PostgreSQL yang terpisah sesuai prinsip Database per Service pada arsitektur microservices. Pendekatan ini bertujuan untuk meningkatkan independensi layanan dan mengurangi ketergantungan antar service. Database *user_db* digunakan untuk menyimpan informasi pengguna, database *task_db* digunakan untuk menyimpan data tugas dan status tugas, sedangkan database *notif_db* digunakan untuk menyimpan data notifikasi. Pemisahan database memungkinkan setiap service mengelola data sesuai kebutuhan bisnisnya masing-masing tanpa mengganggu layanan lain. Selain itu, setiap database menggunakan volume Docker tersendiri sehingga data tetap tersimpan meskipun container dihentikan atau dijalankan kembali. Mekanisme ini mendukung aspek *reliability* dan *persistence* dalam pengelolaan data aplikasi.

3.4.7 Alur Komunikasi Sistem

Alur komunikasi sistem dimulai ketika pengguna mengirimkan permintaan melalui client. Permintaan tersebut diterima oleh API Gateway dan diteruskan menuju service yang sesuai berdasarkan endpoint yang diakses. Apabila pengguna melakukan proses login, permintaan akan diteruskan menuju *User Service*. Setelah proses autentikasi berhasil dilakukan, *User Service* akan menghasilkan token JWT yang kemudian digunakan pada setiap permintaan berikutnya. Ketika user membuat atau mengassign tugas, permintaan akan diteruskan menuju *Task Service*. Setelah data tugas berhasil disimpan, *Task Service* akan mengirimkan informasi kepada *Notification Service* untuk membuat notifikasi baru bagi pengguna yang menerima tugas tersebut. *Notification Service* kemudian menyimpan informasi notifikasi ke dalam database dan menyediakan data tersebut kepada pengguna ketika endpoint notifikasi diakses. Seluruh respons dari service akan dikembalikan kepada client melalui API Gateway sehingga komunikasi tetap terpusat dan terkontrol. Secara keseluruhan, topologi deployment yang dirancang mampu mendukung implementasi arsitektur microservices yang modular, scalable, dan mudah dipelihara. Pemisahan layanan, penggunaan API

Gateway, serta penerapan container Docker memberikan fondasi yang kuat untuk pengembangan sistem pada skala yang lebih besar di masa mendatang.

3.5 Perancangan API (Application Programming Interface)

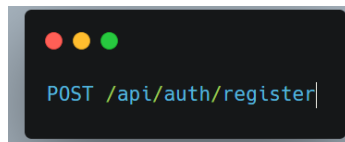
Application Programming Interface (API) merupakan mekanisme komunikasi yang digunakan untuk menghubungkan client dengan layanan *backend* maupun komunikasi antar service dalam arsitektur microservices. Pada sistem yang dikembangkan, API menggunakan pendekatan Representational State Transfer (REST) yang berjalan di atas protokol HTTP. Pemilihan REST API didasarkan pada beberapa pertimbangan, antara lain kemudahan implementasi, sifat stateless, fleksibilitas integrasi, serta kompatibilitas dengan berbagai platform pengembangan. Selain itu, REST API juga menjadi standar yang umum digunakan dalam pengembangan aplikasi modern berbasis layanan. Dalam implementasinya, setiap service menyediakan endpoint yang dapat diakses melalui API Gateway. Dengan pendekatan ini, client tidak perlu mengetahui lokasi atau alamat masing-masing service karena seluruh komunikasi dilakukan melalui satu titik akses yang terpusat.

3.5.1 API User Service

User Service menyediakan layanan autentikasi dan manajemen pengguna.

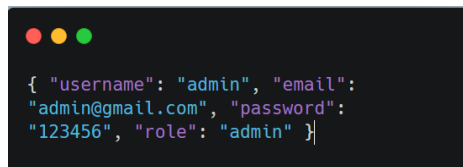
1. Registrasi Pengguna

End point



Fungsi endpoint ini adalah menerima data pengguna baru kemudian menyimpannya ke dalam database setelah password berhasil dienkrpsi menggunakan algoritma bcrypt.

Contoh Request:



2. Login Pengguna

Endpoint:



```
POST /api/auth/login|
```

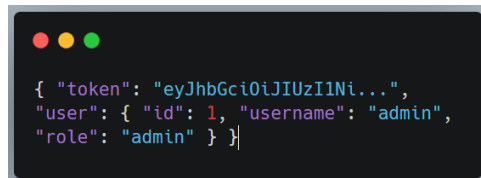
Endpoint ini digunakan untuk melakukan autentikasi pengguna berdasarkan username dan password yang dimasukkan. Jika data valid, sistem akan menghasilkan token JWT yang digunakan untuk mengakses layanan lainnya.

Contoh Request:



```
{ "username": "admin", "password":  
"123456" }
```

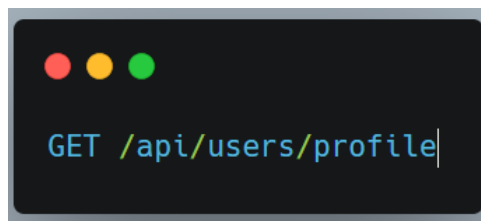
Contoh Response:



```
{ "token": "eyJhbGciOiJIUzI1Ni...",  
  "user": { "id": 1, "username": "admin",  
            "role": "admin" } }
```

3. Profil Pengguna

Endpoint:



```
GET /api/users/profile|
```

Endpoint ini digunakan untuk menampilkan informasi pengguna yang sedang login. Endpoint hanya dapat diakses apabila pengguna menyertakan token JWT yang valid pada header Authorization.

3.5.2 API Task Service

Task Service menyediakan layanan utama yang berkaitan dengan pengelolaan tugas.

1. Menampilkan Daftar Tugas

Endpoint: **GET /api/tasks**

Digunakan untuk menampilkan seluruh tugas yang dimiliki pengguna yang sedang login. Endpoint ini memanfaatkan informasi identitas pengguna yang terdapat pada token JWT untuk menentukan data yang dapat ditampilkan.

2. Membuat Tugas

Endpoint : **POST /api/tasks**

Fungsi endpoint ini adalah membuat tugas baru yang nantinya dapat diberikan kepada pengguna tertentu. Proses ini hanya dapat dilakukan oleh user.

3. Mengubah Data Tugas

Endpoint: **PUT /api/tasks/{id}**

Digunakan untuk memperbarui informasi tugas yang telah dibuat sebelumnya, seperti judul, deskripsi, atau batas waktu penyelesaian.

4. Mengubah status Tugas

Endpoint: **PUT /api/tasks/{id}/status**

Digunakan oleh pengguna untuk memperbarui status penyelesaian tugas sesuai perkembangan pekerjaan yang sedang dilakukan.

3.5.3 API Notification Service

Notification Service menyediakan layanan yang berkaitan dengan penyampaian informasi kepada pengguna.

1. Menampilkan Notifikasi

Endpoint: **GET /api/notifications**

Endpoint digunakan untuk menampilkan seluruh notifikasi yang dimiliki oleh pengguna yang sedang login.

2. Menandai Notifikasi Telah Dibaca

Endpoint: **PATCH /api/notifications/{id}/read**

Mengubah status notifikasi dari belum dibaca menjadi telah dibaca. Dengan mekanisme ini, pengguna dapat membedakan notifikasi baru dan notifikasi yang sudah diperiksa.

3. Endpoint Internal Notifikasi

Endpoint: **POST /notifications**

Endpoint ini tidak diakses secara langsung oleh client, melainkan digunakan sebagai jalur komunikasi internal antar service. Ketika Task Service melakukan assignment tugas, endpoint ini akan menerima data notifikasi yang kemudian disimpan ke dalam database.

3.6 Perancangan JWT Authentication

Keamanan sistem merupakan salah satu aspek penting dalam pengembangan aplikasi berbasis layanan. Untuk memastikan bahwa hanya pengguna yang memiliki hak akses yang dapat menggunakan sistem, diterapkan mekanisme autentikasi menggunakan JSON Web Token (JWT). JWT merupakan standar terbuka yang digunakan untuk mengirimkan informasi identitas pengguna dalam bentuk token yang telah ditandatangani secara digital. Token tersebut berisi informasi pengguna seperti id, username, dan *role* yang dapat digunakan oleh sistem untuk melakukan proses autentikasi maupun otorisasi. Pada saat pengguna berhasil melakukan *login*, *User Service* akan menghasilkan token JWT menggunakan *secret key* yang tersimpan pada *environment variable*. Token tersebut kemudian dikirimkan kepada client dan disimpan untuk digunakan pada setiap permintaan berikutnya.

Alur Autentikasi JWT

- a. Proses autentikasi berlangsung melalui tahapan berikut:
- b. Pengguna memasukkan username dan password.
- c. *User Service* memverifikasi data pengguna.
- d. Password dibandingkan dengan password hash menggunakan bcrypt.
- e. Jika valid, sistem membuat token JWT.
- f. Token dikirimkan kepada client.
- g. Client menyimpan token.
- h. Token dikirim melalui header Authorization pada setiap *request*.
- i. API Gateway memverifikasi validitas token.
- j. *Request* diteruskan menuju service tujuan.

Format header yang digunakan adalah:

Authorization: **Bearer** <token>

Penerapan JWT memberikan beberapa keuntungan, yaitu mengurangi kebutuhan penyimpanan session pada server, meningkatkan performa autentikasi, serta mendukung implementasi sistem yang bersifat stateless.

3.7 Perancangan Docker Compose

Docker Compose digunakan sebagai alat orkestrasi untuk mengelola seluruh container yang terdapat pada sistem. Penggunaan Docker Compose memungkinkan seluruh layanan dijalankan secara otomatis melalui satu file konfigurasi sehingga proses deployment menjadi lebih sederhana dan konsisten. Pada sistem yang dikembangkan, file `docker-compose.yml` mendefinisikan seluruh service, *database*, volume penyimpanan, serta jaringan komunikasi yang digunakan. Seluruh komponen dihubungkan melalui jaringan Docker bernama *task-network* dengan *driver bridge*.

1. Komponen Docker Compose

Komponen yang dijalankan melalui Docker Compose meliputi:

Database Layer

1. user-db
2. task-db
3. notif-db

Ketiga container tersebut menggunakan PostgreSQL versi 15 dan berfungsi sebagai media penyimpanan data untuk masing-masing service.

2. *Service Layer*

1. user-service
2. task-service
3. notification-service

Ketiga layanan tersebut dibangun menggunakan Dockerfile masing-masing dan berjalan secara independen. Setiap service memiliki *environment variable* tersendiri untuk mengakses *database* yang sesuai.

3. Gateway Layer

API Gateway bertugas menerima seluruh permintaan dari client dan meneruskannya ke service yang sesuai. Gateway berjalan pada port 3000 dan menjadi satu-satunya endpoint yang diakses oleh client.

4. Mekanisme Deployment

Proses deployment sistem dilakukan menggunakan perintah: `docker-compose up – build`

Perintah tersebut akan:

1. Membangun image Docker setiap service.
2. Menjalankan seluruh container.
3. Membuat jaringan task-network.
4. Menghubungkan seluruh service ke database masing-masing.
5. Menjalankan API Gateway sebagai pintu masuk utama sistem.

Melalui pendekatan ini, proses instalasi dan deployment menjadi lebih cepat, terstandarisasi, serta mudah direplikasi pada lingkungan pengembangan maupun produksi. Selain itu, Docker Compose juga memudahkan pengelolaan dependensi antar service sehingga keseluruhan sistem dapat berjalan secara terintegrasi.

3.8 Perancangan Dockerfile

Dalam implementasi arsitektur microservices, setiap service dikemas ke dalam container menggunakan Docker. Agar container dapat dibangun secara otomatis dan konsisten, setiap service memerlukan berkas konfigurasi Dockerfile. Dockerfile merupakan sekumpulan instruksi yang digunakan untuk membangun image Docker yang berisi aplikasi beserta seluruh dependensi yang dibutuhkan. Penggunaan Dockerfile memberikan keuntungan berupa konsistensi lingkungan eksekusi pada setiap tahapan pengembangan. Dengan Dockerfile, aplikasi dapat dijalankan dengan konfigurasi yang sama baik pada komputer pengembang, lingkungan pengujian, maupun server produksi. Pendekatan ini membantu mengurangi permasalahan yang sering muncul akibat perbedaan konfigurasi sistem operasi atau versi perangkat lunak yang digunakan.

Pada sistem yang dikembangkan, setiap microservice memiliki Dockerfile tersendiri karena masing-masing service dikembangkan dan dideploy secara independen. Meskipun ketiga service memiliki fungsi bisnis yang berbeda, struktur Dockerfile yang

digunakan relatif serupa karena dibangun menggunakan platform dan *framework* yang sama. Secara umum, proses *build image* dilakukan melalui beberapa tahapan, yaitu menentukan *base image*, menyalin source code ke dalam container, menginstal dependensi aplikasi, melakukan *build* aplikasi, dan menentukan perintah yang dijalankan ketika container aktif.

3.8.1 Struktur Dockerfile User Service

Dockerfile pada User Service bertanggung jawab untuk membangun image yang berisi seluruh komponen autentikasi dan manajemen pengguna.

Tahapan yang dilakukan meliputi:

1. Menggunakan base image Node.js.
2. Menentukan direktori kerja aplikasi.
3. Menyalin file package.json.
4. Menginstal seluruh dependensi aplikasi.
5. Menyalin *source code*.
6. Menjalankan aplikasi pada port yang telah ditentukan.

Dengan pendekatan ini, *User Service* dapat dijalankan secara independen tanpa bergantung pada service lainnya.

3.8.2 Struktur Dockerfile Task Service

Task Service memiliki Dockerfile yang bertugas membangun lingkungan eksekusi untuk seluruh proses pengelolaan tugas. Dockerfile ini memuat seluruh dependensi yang diperlukan untuk menjalankan fitur CRUD tugas, *assignment* tugas, dan pembaruan status tugas. Setelah image berhasil dibangun, container Task Service dapat berkomunikasi dengan database *task_db* melalui jaringan Docker yang telah dikonfigurasi sebelumnya. Dengan demikian, seluruh proses pengelolaan tugas dapat berjalan secara terisolasi dan aman.

3.8.3 Struktur Dockerfile Notification Service

Notification Service menggunakan Dockerfile yang berfungsi membangun image untuk layanan notifikasi. Service ini bertanggung jawab menerima permintaan dari Task Service dan menyimpan informasi notifikasi ke dalam database. Pemisahan

Dockerfile pada setiap service memungkinkan proses deployment dilakukan secara mandiri. Jika terjadi perubahan pada Notification Service, hanya image service tersebut yang perlu dibangun ulang tanpa mempengaruhi service lainnya.

3.9 Kesimpulan Perancangan Sistem

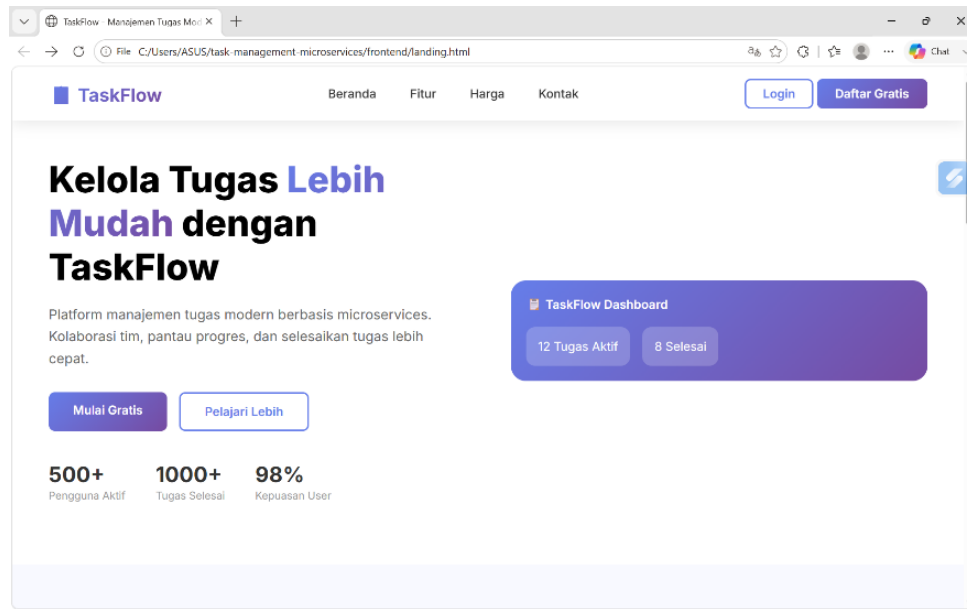
Berdasarkan hasil perancangan yang telah dilakukan, Sistem Manajemen Tugas Berbasis Microservices dirancang dengan memanfaatkan pendekatan arsitektur microservices yang membagi sistem ke dalam tiga layanan utama, yaitu *User Service*, *Task Service*, dan *Notification Service*. Pemisahan layanan dilakukan berdasarkan domain bisnis masing-masing sehingga setiap service dapat dikembangkan, diuji, dan dideploy secara independen. Dari sisi komunikasi, sistem memanfaatkan REST API yang diintegrasikan melalui API Gateway sebagai pintu masuk tunggal seluruh permintaan dari client. Pendekatan ini memberikan kemudahan dalam pengelolaan routing, keamanan, dan integrasi antar layanan. Untuk mendukung aspek keamanan, sistem menerapkan mekanisme autentikasi berbasis JSON Web Token (JWT) yang memungkinkan proses autentikasi dan otorisasi dilakukan secara efisien dan terpusat.

Pada lapisan penyimpanan data, setiap service memiliki database PostgreSQL tersendiri sesuai prinsip Database per Service. Pendekatan ini meningkatkan independensi layanan serta mengurangi ketergantungan antar service. Seluruh komponen sistem dijalankan menggunakan Docker dan Docker Compose sehingga proses deployment dapat dilakukan secara konsisten pada berbagai lingkungan operasional. Secara keseluruhan, rancangan sistem yang dihasilkan telah memenuhi kebutuhan fungsional dan nonfungsional yang telah diidentifikasi pada tahap analisis. Arsitektur yang digunakan juga mendukung prinsip scalability, maintainability, reliability, dan availability yang menjadi karakteristik utama pengembangan perangkat lunak modern berbasis microservices. Rancangan ini selanjutnya menjadi dasar dalam proses implementasi dan pengujian sistem yang akan dibahas pada bab berikutnya.

BAB IV

IMPLEMENTASI DAN PENGUJIAN SISTEM

4.1 Implementasi Sistem



Implementasi sistem merupakan tahap realisasi dari hasil analisis dan perancangan yang telah dilakukan pada bab sebelumnya. Pada tahap ini seluruh rancangan sistem diterjemahkan ke dalam bentuk aplikasi yang dapat dijalankan dan digunakan oleh pengguna. Sistem Manajemen Tugas Berbasis Microservices dikembangkan menggunakan pendekatan arsitektur microservices dengan memanfaatkan teknologi Node.js, Express.js, PostgreSQL, Docker, dan API Gateway. Implementasi dilakukan dengan membagi sistem menjadi tiga layanan utama, yaitu *User Service*, *Task Service*, dan *Notification Service*. Setiap layanan berjalan secara independen dalam container Docker yang berbeda dan memiliki basis data PostgreSQL tersendiri. Seluruh komunikasi antar layanan dilakukan melalui REST API yang dikelola oleh API Gateway sebagai titik masuk utama sistem.

Penerapan arsitektur microservices memungkinkan proses pengembangan dilakukan secara modular sehingga setiap layanan dapat dikembangkan, diuji, dan dipelihara secara terpisah. Selain itu, pendekatan ini juga memberikan fleksibilitas dalam proses deployment karena perubahan pada satu layanan tidak mempengaruhi layanan

lainnya secara langsung. Secara umum implementasi sistem terdiri atas implementasi *User Service*, *Task Service*, *Notification Service*, *PostgreSQL Database*, *API Gateway*, *JWT Authentication*, serta *Docker Compose* sebagai mekanisme deployment aplikasi.

4.2 Implementasi User Service

User Service merupakan layanan yang bertanggung jawab terhadap proses autentikasi dan pengelolaan data pengguna. Service ini dibangun menggunakan *framework* Express.js yang berjalan pada platform Node.js dan terhubung dengan database PostgreSQL melalui koneksi yang telah dikonfigurasi sebelumnya. Pada implementasinya, User Service menyediakan beberapa endpoint utama yang digunakan untuk registrasi pengguna, login, dan pengambilan informasi profil pengguna. Seluruh data pengguna disimpan pada database *user_db* sehingga pengelolaan data pengguna dapat dilakukan secara terpisah dari layanan lainnya.

4.2.1 Implementasi Registrasi Pengguna

Fitur registrasi digunakan untuk menambahkan pengguna baru ke dalam sistem. Pada saat proses registrasi berlangsung, sistem akan menerima data username, email, password, dan role dari pengguna. Password yang diterima tidak langsung disimpan ke database, melainkan terlebih dahulu dienkripsi menggunakan algoritma *bcrypt*. Proses ini dilakukan untuk meningkatkan keamanan data pengguna sehingga password tidak tersimpan dalam bentuk teks biasa. Setelah proses validasi berhasil dilakukan, data pengguna akan disimpan ke dalam tabel *users* dan sistem akan mengembalikan respons bahwa registrasi berhasil dilakukan.

Register - TaskFlow

File C:/Users/ASUS/task-management-microservices/frontend/register.html

TaskFlow

Buat Akun Baru

Mulai kelola tugas dengan lebih baik

Username

mella

Email

mella@gmail.com

Password

Daftar Sekarang

Sudah punya akun? [Login](#)

4.2.2 Implementasi Login

Daftar Sekarang' below it."/>

TaskFlow

Selamat Datang Kembali

Login untuk mengakses dashboard Anda

Email

vaniamelia@gmail.com

Password

Login

Belum punya akun? [Daftar Sekarang](#)

Login digunakan untuk melakukan autentikasi pengguna sebelum mengakses sistem. Pengguna diminta memasukkan username dan password yang telah terdaftar. Sistem akan melakukan pencocokan password menggunakan bcrypt compare. Apabila data yang dimasukkan sesuai dengan data yang terdapat pada database, sistem akan menghasilkan token JWT yang berisi informasi identitas pengguna. Token tersebut

kemudian dikirim kepada client dan digunakan pada setiap permintaan berikutnya melalui *header Authorization*.

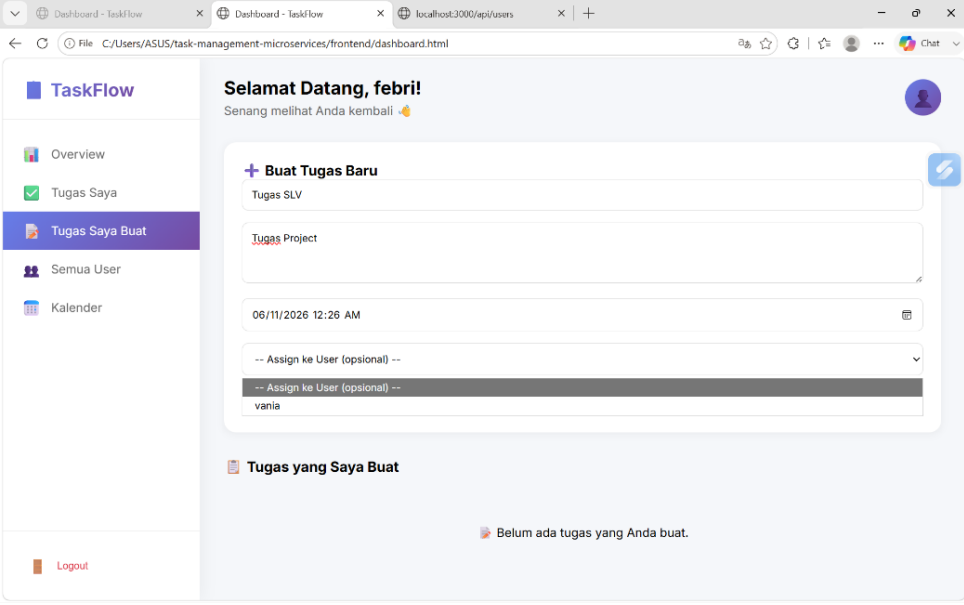
4.2.3 Implementasi Profil Pengguna

Fitur profil digunakan untuk menampilkan informasi pengguna yang sedang login. Data profil diperoleh dari *database* berdasarkan identitas pengguna yang tersimpan pada token JWT. Implementasi fitur ini bertujuan untuk memastikan bahwa pengguna dapat mengakses informasi akunnya sendiri tanpa harus melakukan pencarian data secara manual.

4.3 Implementasi Task Service

Task Service merupakan layanan inti yang bertanggung jawab terhadap pengelolaan tugas dalam sistem. Seluruh aktivitas yang berkaitan dengan tugas dilakukan pada service ini. *Task Service* terhubung dengan database *task_db* yang digunakan untuk menyimpan informasi tugas, status tugas, penerima tugas, dan user yang membuat tugas tersebut.

4.3.1 Implementasi Create Task



The screenshot displays the TaskFlow dashboard interface. On the left, a sidebar menu includes 'Overview', 'Tugas Saya', 'Tugas Saya Buat' (highlighted), 'Semua User', and 'Kalender'. The main content area features a welcome message 'Selamat Datang, febri!' and a 'Buat Tugas Baru' (Create New Task) form. The form contains input fields for 'Tugas SLV', 'Tugas Project', and a date/time field set to '06/11/2026 12:26 AM'. Below these is a dropdown menu for assigning a user, currently showing 'vania'. At the bottom, a section titled 'Tugas yang Saya Buat' (Tasks I Created) displays the message 'Belum ada tugas yang Anda buat.' (No tasks created yet).

Fitur Create Task digunakan oleh user untuk membuat tugas baru. User mengisi informasi seperti:

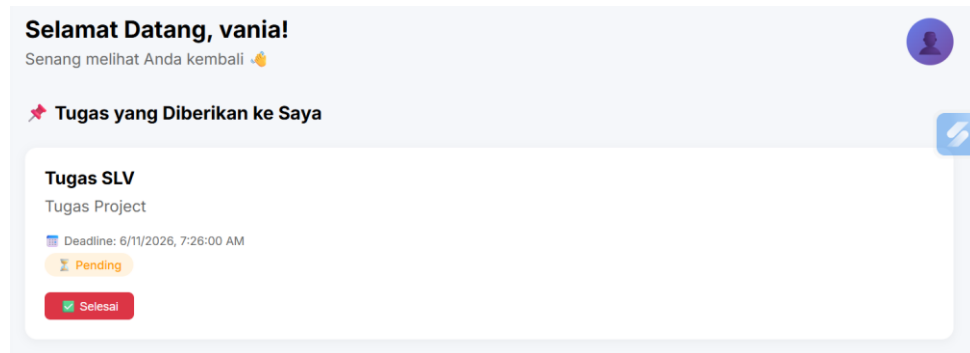
1. Judul tugas

2. Deskripsi tugas
3. Deadline tugas
4. Pengguna yang dituju

Setelah data berhasil divalidasi, sistem akan menyimpan informasi tersebut ke dalam tabel tasks.

4.3.2 Implementasi Read Task

Fitur Read Task digunakan untuk menampilkan daftar tugas. User dapat melihat seluruh tugas yang tersedia dalam sistem, sedangkan pengguna hanya dapat melihat tugas yang diberikan kepadanya. Pembatasan akses ini dilakukan berdasarkan identitas dan role yang terdapat pada token JWT.



4.3.3 Implementasi Update Task

Update Task digunakan untuk mengubah informasi tugas yang telah dibuat sebelumnya.

User dapat memperbarui:

1. Judul tugas
2. Deskripsi tugas
3. Deadline
4. Penerima tugas

Setiap perubahan akan tersimpan pada database dan memperbarui nilai `updated_at`.

Assign ke User (opsional) --

 **Edit Tugas**

Tugas dari CMD

Ini tugas

-- Pilih User --

mm/dd/yyyy -- : --



 Pending ▾

Simpan Perubahan

4.3.4 Implementasi Delete Task

Delete Task digunakan untuk menghapus tugas yang sudah tidak diperlukan lagi. Ketika user melakukan penghapusan tugas, sistem akan menghapus data yang sesuai dari database task_db.

 **Tugas yang Saya Buat**

Tugas dari CMD

Ini tugas

Assign ke: Belum di-assign

 Pending

 Edit

 Hapus

4.3.5 Implementasi Assign Task

Assign Task merupakan fitur yang digunakan untuk memberikan tugas kepada pengguna tertentu. Setelah assignment berhasil dilakukan, Task Service akan mengirimkan request ke Notification Service untuk membuat notifikasi baru bagi pengguna yang menerima tugas. Implementasi ini menunjukkan penerapan komunikasi antar service dalam arsitektur microservices.

+

Buat Tugas Baru

Metopen

membuat proposal dari bab 1 sampai bab 3

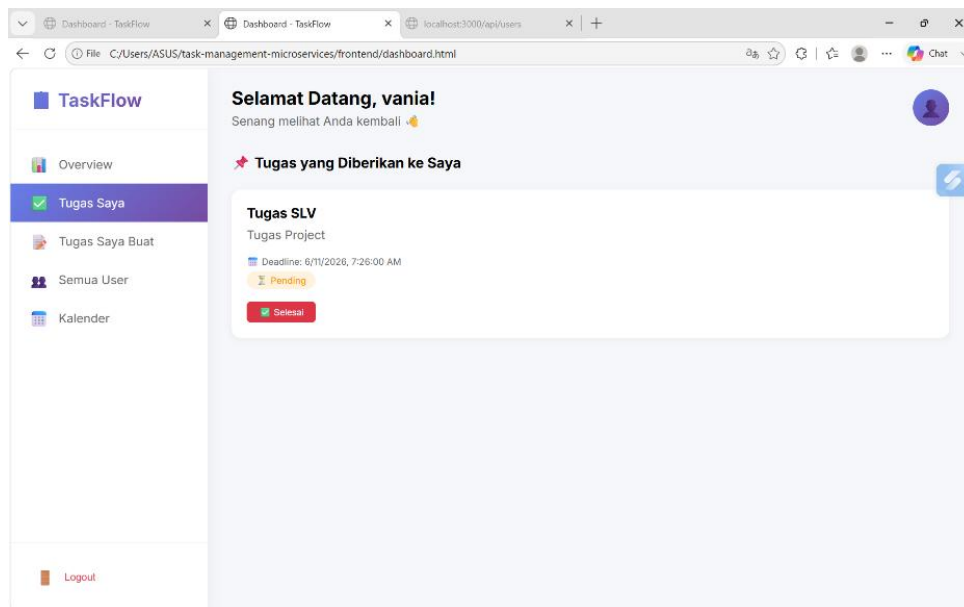
06/12/2026 09:29 PM

-- Assign ke User (opsional) --

-- Assign ke User (opsional) --

febri

4.3.6 Implementasi Update Status Task



Pengguna dapat memperbarui status tugas sesuai progres pekerjaan yang sedang dilakukan.

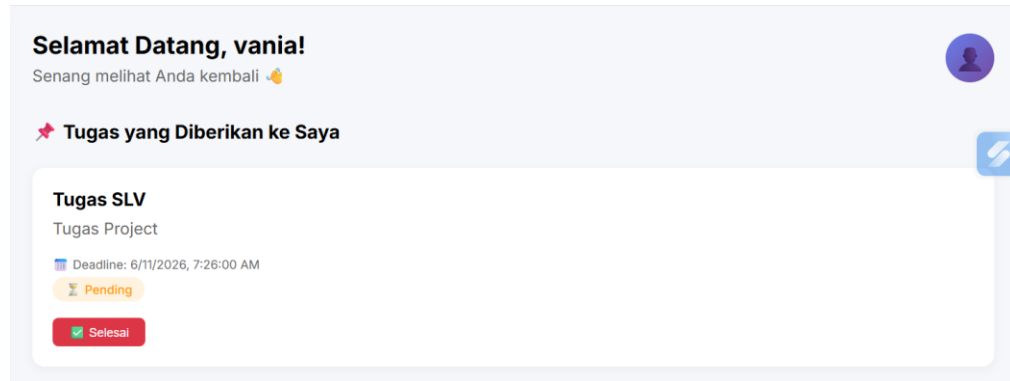
Status yang tersedia meliputi:

1. *Pending*
2. *In Progress*
3. *Completed*

Perubahan status akan langsung tersimpan ke database dan dapat dipantau oleh user.

4.4 Implementasi Notification Service

Notification Service bertanggung jawab terhadap pengelolaan notifikasi pengguna. Layanan ini menerima data dari Task Service ketika terjadi assignment tugas dan menyimpannya ke database notif_db.



4.4.1 Implementasi Pembuatan Notifikasi

Ketika user memberikan tugas kepada pengguna tertentu, *Task Service* akan mengirimkan data notifikasi melalui REST API. Notification Service kemudian:

1. Menerima request.
2. Memvalidasi data.
3. Menyimpan data ke tabel notifications.
4. Mengembalikan respons berhasil.

Dengan mekanisme ini, pengguna dapat mengetahui adanya tugas baru secara cepat.

4.4.2 Implementasi Menampilkan Notifikasi

Fitur ini digunakan untuk menampilkan seluruh notifikasi yang dimiliki oleh pengguna. Data diambil dari database notif_db berdasarkan user_id yang terdapat pada token JWT.

4.4.3 Implementasi Menandai Notifikasi Telah Dibaca

Pengguna dapat menandai notifikasi sebagai telah dibaca. Proses ini dilakukan dengan memperbarui nilai kolom is_read menjadi TRUE pada database.

4.5 Implementasi PostgreSQL Database

Sistem menggunakan PostgreSQL sebagai *Database Management System* (DBMS) utama. PostgreSQL dipilih karena memiliki performa yang baik, mendukung transaksi ACID, bersifat *open source*, serta mampu menangani kebutuhan aplikasi modern berbasis microservices. Setiap service memiliki database yang berbeda, yaitu:

Service	Database
User Service	User_db
Task service	Task_db
Notification Service	Notif_db

```
C:\Users\ASUS>docker exec -it user_db psql -U postgres -d user_db
b -c "\dt"
      List of relations
Schema | Name  | Type  | Owner
-----+-----+-----+-----
public | tasks | table | postgres
public | users | table | postgres
(2 rows)
```

Pengecekan daftar tabel pada database user_db menggunakan perintah `docker exec -it user_db psql -U postgres -d user_db -c "\dt"`. Hasil output menunjukkan bahwa database user_db memiliki dua tabel dalam skema public, yaitu tabel tasks dan tabel users, yang keduanya dimiliki oleh user postgres.

```
Table "public.users"
Column | Type                      | Collation | Nullable | Default
-----+-----+-----+-----+-----
id      | integer                   |           | not null | nextval('users_id_seq'::regclass)
username | character varying(100)   |           | not null |
email   | character varying(255)   |           | not null |
password | character varying(255)   |           | not null |
created_at | timestamp without time zone |           |          | CURRENT_TIMESTAMP
Indexes:
"users_pkey" PRIMARY KEY, btree (id)
"users_email_key" UNIQUE CONSTRAINT, btree (email)
```

Struktur detail tabel users pada database user_db. Tabel ini memiliki lima kolom, yaitu id bertipe integer dengan nilai auto-increment sebagai primary key, username bertipe varchar(100), email bertipe varchar(255) dengan constraint unique sehingga tidak boleh ada data yang sama, password bertipe varchar(255), serta created_at bertipe timestamp yang secara otomatis terisi dengan waktu saat data dibuat. Tabel users juga memiliki dua

index, yaitu `users_pkey` sebagai primary key dan `users_email_key` sebagai unique constraint pada kolom email.

```
C:\Users\ASUS>docker exec -it task_db psql -U postgres -d task_db -c "\dt"
List of relations
Schema | Name   | Type  | Owner
-----+-----+-----+-----
public | tasks  | table | postgres
(1 row)
```

Pengecekan daftar tabel pada database `task_db` menggunakan perintah `docker exec -it task_db psql -U postgres -d task_db -c "\dt"`. Hasil output menunjukkan bahwa database `task_db` hanya memiliki satu tabel, yaitu tabel `tasks`. Hal ini menunjukkan bahwa layanan tugas memiliki database tersendiri yang terpisah dari database pengguna, sesuai dengan prinsip pemisahan database dalam arsitektur microservices.

Column	Type	Table "public.tasks"	Collation	Nullable	Default
id	integer			not null	nextval('tasks_id_seq'::regclass)
title	character varying(255)			not null	
description	text				
status	character varying(50)				'pending'::character varying
assigned_to	integer				
created_by	integer			not null	
deadline	timestamp without time zone				
reminder_sent	boolean				false
created_at	timestamp without time zone				CURRENT_TIMESTAMP
updated_at	timestamp without time zone				CURRENT_TIMESTAMP

Indexes:
"tasks_pkey" PRIMARY KEY, btree (id)

Pengecekan daftar tabel pada database `task_db` menggunakan perintah `docker exec -it task_db psql -U postgres -d task_db -c "\dt"`. Hasil output menunjukkan bahwa database `task_db` hanya memiliki satu tabel, yaitu tabel `tasks`. Hal ini menunjukkan bahwa layanan tugas memiliki database tersendiri yang terpisah dari database pengguna, sesuai dengan prinsip pemisahan database dalam arsitektur microservices.

```
C:\Users\ASUS>docker exec -it notif_db psql -U postgres -d notif_db -c "\dt"
List of relations
Schema | Name       | Type  | Owner
-----+-----+-----+-----
public | notifications | table | postgres
(1 row)
```

Daftar tabel pada database notif_db menggunakan perintah docker exec -it notif_db psql -U postgres -d notif_db -c "\dt". Hasil output menunjukkan bahwa database notif_db hanya memiliki satu tabel, yaitu tabel notifications, yang dimiliki oleh user postgres. Hal ini menunjukkan bahwa layanan notifikasi memiliki database tersendiri yang terpisah dari layanan lainnya.

```
C:\Users\ASUS>docker exec -it notif_db psql -U postgres -d notif_db -c "\dt notifications"
Table "public.notifications"
  Column      |      Type      | Collation | Nullable |      Default
-----|-----|-----|-----|-----
id            | integer        |           | not null | nextval('notifications_id_seq'::regclass)
user_id       | integer        |           | not null |
title         | character varying(255) |           | not null |
message       | text           |           | not null |
is_read       | boolean        |           |         | false
created_at    | timestamp without time zone |           |         | CURRENT_TIMESTAMP
Indexes:
  "notifications_pkey" PRIMARY KEY, btree (id)

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug notif\_db
  Learn more at https://docs.docker.com/go/debug-cli/
```

Struktur detail tabel notifications pada database notif_db. Tabel ini memiliki tujuh kolom, yaitu id bertipe integer dengan nilai auto-increment sebagai primary key, user_id bertipe integer yang bersifat not null sebagai referensi ke pengguna penerima notifikasi, title bertipe varchar(255) yang bersifat not null, message bertipe text yang bersifat not null untuk menyimpan isi pesan notifikasi, is_read bertipe boolean dengan nilai default false yang menandakan apakah notifikasi sudah dibaca atau belum, serta created_at bertipe timestamp yang otomatis terisi dengan waktu saat notifikasi dibuat. Tabel ini memiliki satu index yaitu notifications_pkey sebagai primary key pada kolom id.

Struktur tabel user

Kolom	Tipe Data	Keterangan
Id	SERIAL (Primary Key)	ID unik user
username	VARCHAR(100)	Nama pengguna
Email	VARCHAR(255) UNIQUE	Email user
password	VARCHAR(255)	Password terenkripsi
created_at	TIMESTAMP	Waktu registrasi

Struktur Tabel task

Kolom	Tipe Data	Keterangan
id	SERIAL (PK)	ID unik tugas
title	VARCHAR(255)	Judul tugas
description	TEXT	Deskripsi tugas
status	VARCHAR(50)	pending/done/expired
assigned_to	INTEGER (FK)	ID user penerima
created_by	INTEGER (FK)	ID pembuat tugas
deadline	TIMESTAMP	Batas waktu tugas
reminder_sent	BOOLEAN	Status reminder
created_at	TIMESTAMP	Waktu dibuat
updated_at	TIMESTAMP	Waktu update

Struktur Tabel Notification

Kolom	Tipe Data	Keterangan
id	SERIAL (PK)	ID notifikasi
user_id	INTEGER (FK)	ID penerima
title	VARCHAR(255)	Judul notifikasi
message	TEXT	Isi notifikasi
is_read	BOOLEAN	Status baca
created_at	TIMESTAMP	Waktu dibuat

Pemisahan database dilakukan untuk menjaga independensi layanan serta mengurangi ketergantungan antar service. Seluruh database dijalankan menggunakan container PostgreSQL yang terpisah dan terhubung melalui jaringan Docker.

4.6 Implementasi API Gateway

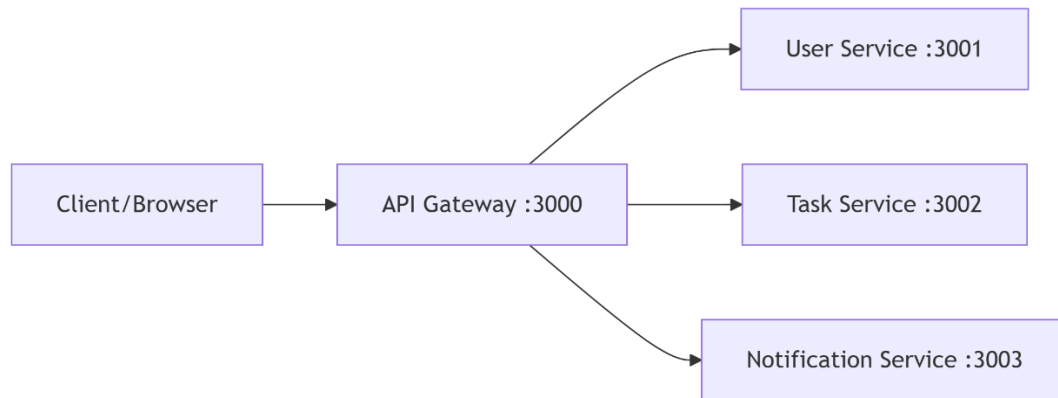
API Gateway merupakan komponen yang berfungsi sebagai pintu masuk utama seluruh permintaan dari client. Pada implementasinya, API Gateway bertanggung jawab terhadap:

1. *Request routing*
2. *Reverse proxy*
3. *JWT validation*
4. *Request forwarding*

Seluruh endpoint yang diakses pengguna akan diteruskan menuju service yang sesuai.

No	Endpoint	Method	Service Tujuan	Port
1	/api/users/*	GET, POST, PUT	User Service	3001
2	/api/auth/*	POST	User Service	3001
3	/api/tasks/*	GET, POST, PUT, DELETE	Task Service	3002
4	/api/notifications/*	GET, POST, PUT	Notification Service	3003

```
42 // ROUTING
43 app.use('/api/users', (req, res) => forwardRequest(req, res, USER_SERVICE));
44 app.use('/api/auth', (req, res) => forwardRequest(req, res, USER_SERVICE));
45 app.use('/api/tasks', (req, res) => forwardRequest(req, res, TASK_SERVICE));
46 app.use('/api/notifications', (req, res) => forwardRequest(req, res, NOTIF_SERVICE));
```



Alur komunikasi sistem secara keseluruhan, di mana Client/Browser mengirimkan request ke API Gateway pada port 3000, kemudian API Gateway meneruskan request tersebut ke masing-masing layanan sesuai dengan endpoint yang dituju, yaitu User Service, Task Service, atau Notification Service. Pendekatan ini merupakan penerapan pola API Gateway dalam arsitektur microservices, di mana seluruh request dari client dikelola melalui satu titik masuk tunggal sebelum didistribusikan ke layanan-layanan yang bersangkutan. Dengan mekanisme ini, client tidak perlu mengetahui lokasi masing-masing service..

4.7 Implementasi JWT Authentication

JWT Authentication digunakan untuk memastikan bahwa hanya pengguna yang memiliki hak akses yang dapat menggunakan layanan sistem. Proses autentikasi dimulai ketika pengguna melakukan login dan berhasil diverifikasi oleh *User Service*. Setelah autentikasi berhasil, sistem menghasilkan token JWT yang berisi:

1. *User ID*
2. *Username*
3. *Role*
4. *Expiration Time*

```
1 {
2   "message": "Login berhasil",
3   "user": {
4     "id": 1,
5     "username": "vania",
6     "email": "vaniamelia@gmail.com"
7   },
8   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
9     .eyJpZCI6MSwiZW1haWwiOiJ2YW5pYW1lbG1hQGdtYWlsLmNvbSI6Im1hdCI6MTc4MTE4OTgyNiwiZXhwIjo
     xNzg5Nzk0NjI2fQ.ZCTPr9wYzIt9V6hi4dD70Sr9V4zzKm_BrMxhh4DoVDo"
10 }
```

Token tersebut digunakan pada setiap request melalui header: “Authorization: Bearer <token>” Pada saat request diterima, API Gateway akan memverifikasi token tersebut sebelum meneruskannya ke service tujuan. Mekanisme ini, sistem dapat menerapkan autentikasi dan otorisasi secara aman dan efisien.

Apabila password salah maka akan menampilkan pesan error.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/api/users/login
- Body:** JSON format containing `{ "email": "vaniamelia@gmail.com", "password": "salah" }`
- Status:** 401 Unauthorized
- Size:** 37 Bytes
- Time:** 160 ms
- Response Body:** JSON format containing `{ "error": "Email atau password salah" }`

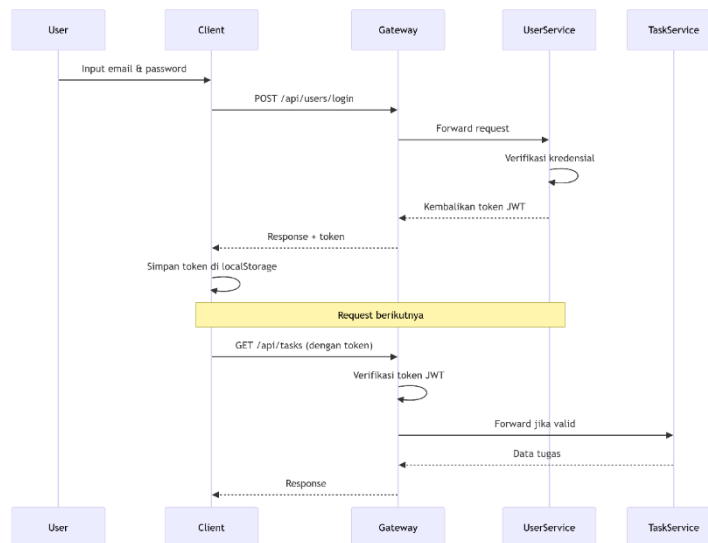
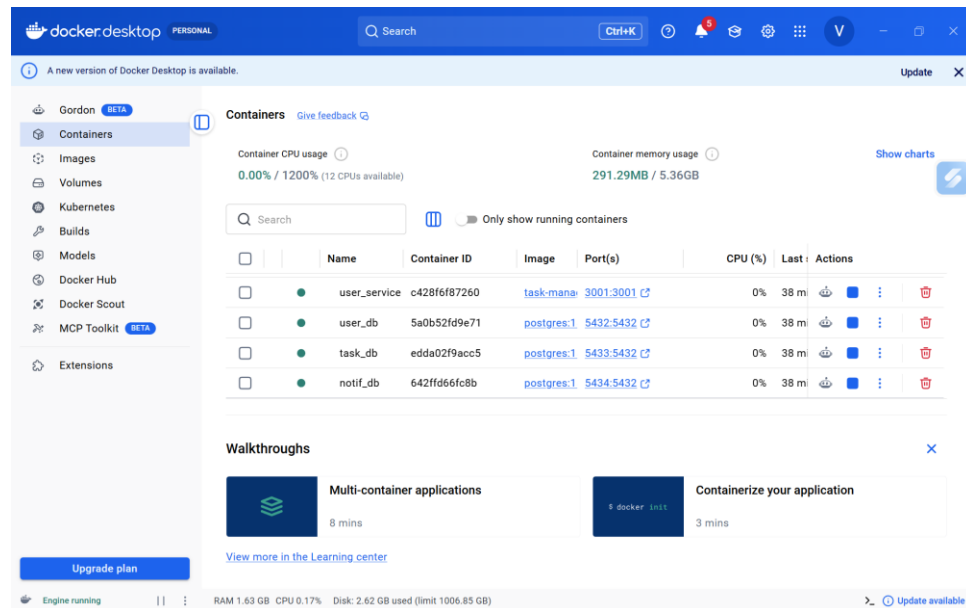


Diagram yang menggambarkan alur autentikasi dan otorisasi pada sistem. Diagram ini melibatkan lima komponen utama, yaitu User, Client, Gateway, UserService, dan TaskService. Alur dimulai ketika User memasukkan email dan password pada Client, kemudian Client mengirimkan request POST /api/users/login ke Gateway. Gateway meneruskan request tersebut ke UserService, yang kemudian melakukan verifikasi kredensial secara internal. Setelah kredensial berhasil diverifikasi, UserService mengembalikan token JWT ke Gateway, lalu Gateway meneruskan response beserta token tersebut ke Client. Token JWT kemudian disimpan oleh Client di localStorage untuk digunakan pada request berikutnya. Pada tahap request berikutnya, Client mengirimkan request GET /api/tasks dengan menyertakan token JWT ke Gateway. Gateway terlebih dahulu melakukan verifikasi token JWT melalui UserService. Apabila token dinyatakan valid, Gateway meneruskan request ke TaskService untuk mengambil data tugas. TaskService kemudian mengembalikan data tugas ke Gateway, dan Gateway meneruskan response tersebut ke Client. Alur ini menunjukkan penerapan mekanisme autentikasi berbasis JWT (JSON Web Token) dalam arsitektur microservices, di mana token digunakan sebagai bukti identitas pengguna yang diverifikasi oleh Gateway sebelum request diteruskan ke layanan yang dituju.

4.8 Implementasi Docker Compose

Docker Compose digunakan untuk mengelola seluruh container yang terdapat pada sistem. Melalui file docker-compose.yml, seluruh service dapat dijalankan secara otomatis menggunakan satu perintah.

“docker-compose up –build”



Container yang dijalankan meliputi:

1. API Gateway
2. User Service
3. Task Service
4. Notification Service
5. PostgreSQL User Database
6. PostgreSQL Task Database
7. PostgreSQL Notification Database

Seluruh container terhubung melalui jaringan Docker bernama task-network sehingga dapat saling berkomunikasi dengan mudah.

```
C:\Users\ASUS\task-management-microservices>docker compose ps
time="2026-06-11T22:04:02+07:00" level=warning msg="C:\Users\ASUS\task-management-microservices\docker-compose.yml: the attribute
'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                PORTS                COMMAND                SERVICE                CREATED
api_gateway         task-management-microservices-api-gateway         "docker-entrypoint.s..." api-gateway         4 days ag
o Up 41 minutes     0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
notif_db            postgres:15
o Up 41 minutes (healthy) 0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp "docker-entrypoint.s..." notif-db            4 days ag
notification_service task-management-microservices-notification-service "docker-entrypoint.s..." notification-service 4 days ag
o Up 41 minutes     0.0.0.0:3003->3003/tcp, [::]:3003->3003/tcp
task_db             postgres:15
o Up 41 minutes (healthy) 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp "docker-entrypoint.s..." task-db             4 days ag
task_service        task-management-microservices-task-service        "docker-entrypoint.s..." task-service        4 days ag
o Up 41 minutes     0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
user_db             postgres:15
o Up 41 minutes (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp "docker-entrypoint.s..." user-db             4 days ag
user_service        task-management-microservices-user-service        "docker-entrypoint.s..." user-service        4 days ag
o Up 41 minutes     0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
C:\Users\ASUS\task-management-microservices>
```

Hasil monitoring infrastruktur Docker Compose menunjukkan bahwa semua layanan microservices dalam sistem berjalan dengan normal. Pada tanggal 11 Juni 2026 pukul 22:08 WIB, terdapat 9 container aktif yang beroperasi dengan status "running". Sistem terdiri dari beberapa komponen utama yaitu:

1. **API Gateway** yang beroperasi sebagai gateway utama dengan mapping port 3000
2. **Database Services** untuk task dan user yang menggunakan PostgreSQL
3. **Microservices** meliputi notification service, task service, dan user service dengan masing-masing mapping port yang sesuai

Semua port mapping telah dikonfigurasi dengan baik dan tidak ada error atau warning yang terdeteksi. Secara keseluruhan, infrastruktur Docker Compose dalam keadaan operasional dan siap melayani kebutuhan sistem.

4.9 Pengujian Sistem

Pengujian dilakukan menggunakan Postman untuk memastikan seluruh endpoint dan fungsi sistem berjalan sesuai kebutuhan.

No.	Skenario Pengujian	Input	Hasil yang diharapkan	Hasil Aktual	Status
1	Registrasi pengguna	Data pengguna valid	Data tersimpan	Berhasil	Valid
2	Login pengguna	Username dan password valid	JWT berhasil dibuat	Berhasil	Valid
3	Login gagal	Password salah	Password salah	Berhasil	Valid

4	Melihat profil pengguna	Token valid	Data profil tampil	Berhasil	Valid
5	Membuat tugas	Data tugas valid	Tugas tersimpan	Berhasil	Valid
6	Melihat daftar tugas	Token valid	Daftar tugas tampil	Berhasil	Valid
7	Mengubah tugas	Data perubahan valid	Data diperbarui	Berhasil	Valid
8	Menghapus tugas	ID tugas valid	Data terhapus	Berhasil	Valid
9	Assign tugas	User valid	Tugas terassign	Berhasil	Valid
10	Membuat notifikasi	Assignment berhasil	Notifikasi tersimpan	Berhasil	Valid
11	Melihat notifikasi	Token valid	Data tampil	Berhasil	Valid
12	Menandai notifikasi dibaca	ID valid	Status berubah	Berhasil	Valid
13	Akses endpoint tanpa token	Tidak ada token	Unauthorized	Berhasil	Valid
14	Token tidak valid	Token salah	Unauthorized	Berhasil	Valid
15	Komunikasi Task Service ke Notification Service	Assign task	Unauthorized	Berhasil	Valid

4.10 Analisis Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan, seluruh fungsi utama sistem berhasil berjalan sesuai dengan kebutuhan yang telah ditentukan pada tahap analisis dan perancangan. Pengujian menunjukkan bahwa setiap service mampu menjalankan tanggung jawabnya secara independen tanpa mengganggu layanan lainnya. Dari aspek fungsionalitas, User Service berhasil menangani proses registrasi dan autentikasi pengguna menggunakan JWT. Task Service mampu mengelola seluruh proses CRUD tugas serta assignment tugas kepada pengguna. Notification Service juga berhasil menerima data dari

Task Service dan menghasilkan notifikasi secara otomatis. Dari aspek integrasi, komunikasi antar service menggunakan REST API berjalan dengan baik. Proses assignment tugas berhasil memicu pembuatan notifikasi tanpa mengalami kegagalan komunikasi antar layanan. Hal ini menunjukkan bahwa implementasi microservices telah berjalan sesuai dengan rancangan arsitektur yang dibuat. Dari aspek deployment, Docker Compose berhasil menjalankan seluruh container secara bersamaan dan menjaga konektivitas antar service melalui jaringan Docker yang telah dikonfigurasi. Penggunaan container juga mempermudah proses pengujian dan deployment sistem. Secara keseluruhan, tingkat keberhasilan pengujian mencapai 100% terhadap seluruh skenario yang diuji. Hasil tersebut menunjukkan bahwa Sistem Manajemen Tugas Berbasis Microservices telah memenuhi kebutuhan fungsional dan siap digunakan sebagai solusi pengelolaan tugas yang lebih terstruktur, modular, dan mudah dikembangkan pada masa mendatang.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil analisis, perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa Sistem Manajemen Tugas Berbasis Microservices berhasil dikembangkan sesuai dengan kebutuhan yang telah ditetapkan. Sistem dirancang menggunakan pendekatan arsitektur microservices yang membagi aplikasi menjadi beberapa layanan independen, yaitu *User Service*, *Task Service*, dan *Notification Service*. Pemisahan layanan berdasarkan domain bisnis memungkinkan setiap service memiliki tanggung jawab yang jelas sehingga pengembangan dan pemeliharaan sistem dapat dilakukan secara lebih efektif.

Implementasi API Gateway sebagai pintu masuk utama sistem berhasil menyederhanakan proses komunikasi antara client dan layanan backend. Selain itu, penerapan JSON Web Token (JWT) sebagai mekanisme autentikasi dan otorisasi mampu meningkatkan keamanan sistem dengan memastikan bahwa hanya pengguna yang memiliki hak akses yang dapat menggunakan layanan yang tersedia. Penggunaan PostgreSQL sebagai basis data pada masing-masing service juga mendukung prinsip Database per Service yang menjadi karakteristik utama arsitektur microservices.

Hasil pengujian menunjukkan bahwa seluruh fitur utama sistem dapat berjalan sesuai dengan kebutuhan fungsional yang telah dirancang. Proses registrasi pengguna, login, pengelolaan tugas, assignment tugas, pembuatan notifikasi, serta komunikasi antar service berhasil dijalankan tanpa mengalami kendala yang signifikan. Integrasi antar layanan melalui REST API juga berjalan dengan baik sehingga setiap proses bisnis dapat diselesaikan secara otomatis dan terstruktur.

Dari sisi infrastruktur, penggunaan Docker dan Docker Compose berhasil mendukung proses deployment sistem secara konsisten dan efisien. Seluruh layanan dapat dijalankan dalam container yang terisolasi namun tetap mampu berkomunikasi melalui jaringan yang sama. Pendekatan ini memberikan kemudahan dalam proses pengembangan, pengujian, maupun implementasi sistem pada lingkungan yang berbeda.

Secara keseluruhan, Sistem Manajemen Tugas Berbasis Microservices yang dibangun telah memenuhi tujuan proyek, yaitu menyediakan solusi pengelolaan tugas yang terstruktur, aman, mudah dikembangkan, dan mampu mendukung kebutuhan kolaborasi pengguna dalam suatu organisasi atau tim kerja. Selain itu, sistem yang dihasilkan juga telah menerapkan konsep-konsep modern dalam rekayasa perangkat lunak seperti microservices, containerization, REST API, dan authentication berbasis token.

5.2 Saran

Meskipun sistem yang dikembangkan telah mampu memenuhi kebutuhan dasar pengelolaan tugas, masih terdapat beberapa aspek yang dapat dikembangkan lebih lanjut untuk meningkatkan kualitas dan skalabilitas sistem di masa mendatang. Pertama, komunikasi antar service yang saat ini masih menggunakan mekanisme synchronous REST API dapat dikembangkan menggunakan teknologi Message Queue seperti RabbitMQ atau Apache Kafka. Pendekatan ini dapat meningkatkan keandalan sistem, terutama ketika jumlah transaksi dan komunikasi antar layanan semakin besar. Kedua, deployment sistem yang saat ini menggunakan Docker Compose dapat ditingkatkan dengan mengimplementasikan Kubernetes sebagai platform orkestrasi container. Penggunaan Kubernetes memungkinkan proses *scaling*, *load balancing*, dan manajemen container dilakukan secara otomatis sehingga sistem lebih siap digunakan pada lingkungan produksi dengan jumlah pengguna yang tinggi. Ketiga, perlu ditambahkan mekanisme monitoring dan observability menggunakan tools seperti Prometheus dan Grafana. Implementasi monitoring akan membantu user dalam memantau performa layanan, penggunaan sumber daya, serta mendeteksi gangguan sistem secara lebih cepat. Keempat, sistem dapat dikembangkan dengan menerapkan centralized logging menggunakan ELK Stack atau solusi serupa. Dengan adanya logging terpusat, proses analisis kesalahan, audit aktivitas, dan pemeliharaan sistem akan menjadi lebih mudah dan efisien. Kelima, untuk meningkatkan kualitas proses pengembangan perangkat lunak, disarankan menerapkan mekanisme Continuous Integration dan Continuous Deployment (CI/CD) menggunakan platform seperti GitHub Actions atau Jenkins. Implementasi CI/CD akan membantu mempercepat proses pengujian, integrasi kode, dan deployment aplikasi secara otomatis. Dengan pengembangan tersebut, sistem diharapkan dapat menjadi lebih andal,

skalabel, aman, dan mampu memenuhi kebutuhan organisasi yang lebih kompleks pada masa yang akan datang.

DAFTAR PUSTAKA

- Newman, Sam. (2021). *Building Microservices: Flexible Software Architecture* (2nd ed.). Sebastopol, CA: O'Reilly Media.
- Richardson, Chris. (2019). *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications.
- Fielding, Roy Thomas. (2000). *Architectural Styles and the Design of Network-Based Software Architectures*. Doctoral Dissertation, University of California, Irvine.
- Martin, Robert C.. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Pearson Education.
- Fowler, Martin. (2018). *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley Professional.
- Docker Documentation. (2025). Docker Official Documentation. Diakses pada 10 Juni 2026.
- Docker Compose Documentation. (2025). Docker Compose Documentation. Diakses pada 10 Juni 2026.
- PostgreSQL Documentation. (2025). PostgreSQL Official Documentation. Diakses pada 10 Juni 2026.
- JWT Official Documentation. (2025). Introduction to JSON Web Tokens. Diakses pada 10 Juni 2026.
- Express.js Documentation. (2025). Express.js Official Documentation. Diakses pada 10 Juni 2026.
- Node.js Documentation. (2025). Node.js Official Documentation. Diakses pada 10 Juni 2026.
- Bass, Len., Clements, Paul., & Kazman, Rick. (2022). *Software Architecture in Practice* (4th ed.). Boston: Addison-Wesley Professional.
- Pressman, Roger S., & Maxim, Bruce R.. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). New York: McGraw-Hill Education.
- Sommerville, Ian. (2019). *Software Engineering* (10th ed.). Boston: Pearson Education.